Fleet Telematics Lakehouse on Microsoft Fabric

Project Documentation

SIC 8 / 2026, Big Data Engineering, Project 3

Fabric workspace: SIC8-Fleet-Telematics

Project repository: https://github.com/shoman042/fleet-telematics-fabric

Source documentation last updated: 28 October 2026

1. Introduction

1.1 Project Overview

The Fleet Telematics Lakehouse is a big data engineering project implemented on Microsoft Fabric as part of SIC 8 / 2026 (Big Data Engineering, Project 3). The project processes simulated telemetry from a fleet of trucks. Four signals (GPS, engine, fuel and temperature) and a device status feed are generated by a seeded Python simulator, streamed through Fabric Eventstreams, stored in raw form, and refined through Bronze, Silver and Gold layers. A real-time layer produces alerts, and a serving layer exposes the Gold tables, alerts and live status to Power BI dashboards.

The simulator injects known anomalies, data faults, late arrivals and offline windows and records a ground truth for each of them. This allows every stage of the pipeline to be reconciled against the injected events.

1.2 Objectives

The objectives documented in the source material are as follows.

- **Phase 1 (Foundation and Data Generator):** create the environment the pipeline will process, namely the Fabric workspace, all reference data, a reproducible truck simulation that streams the signals, the injected problems with their ground truth, and a shared library. The behaviour of the Eventstream is verified first (Day-1 checks) so that all later work rests on confirmed facts.
- **Silver layer:** convert the raw telemetry stored in Bronze into clean, typed and enriched Silver tables, rejecting invalid messages to a dead-letter queue (DLQ) and applying a quality gate.
- **Gold layer:** convert Silver data into dimensions, fact tables and dashboard tables that answer business questions.
- **Orchestration:** automate the batch steps in the order Bronze, Silver, Alerts, Gold, with an automatic schedule that runs twice a day.

1.3 Document Scope and Status

This document consolidates the Phase 1 documentation (last updated 26 October 2026), the Silver layer notebook documentation (28 October 2026), the Gold layer guide, the real-time processing and dashboard documentation, and the batch orchestration documentation (28 October 2026). Information that was not provided in the source material is not included.

Phase 1 tasks P1-1 to P1-11 are complete and were verified locally. Task P1-12 (offline windows) is implemented and switched off by default; its local verification is pending. The Fabric-side checks listed in Section 14.4 had not yet been recorded at the time of the source documentation.

2. System Architecture

2.1 Architectural Overview

Figure 1 presents the architecture of the project. The components shown in the diagram are described below.

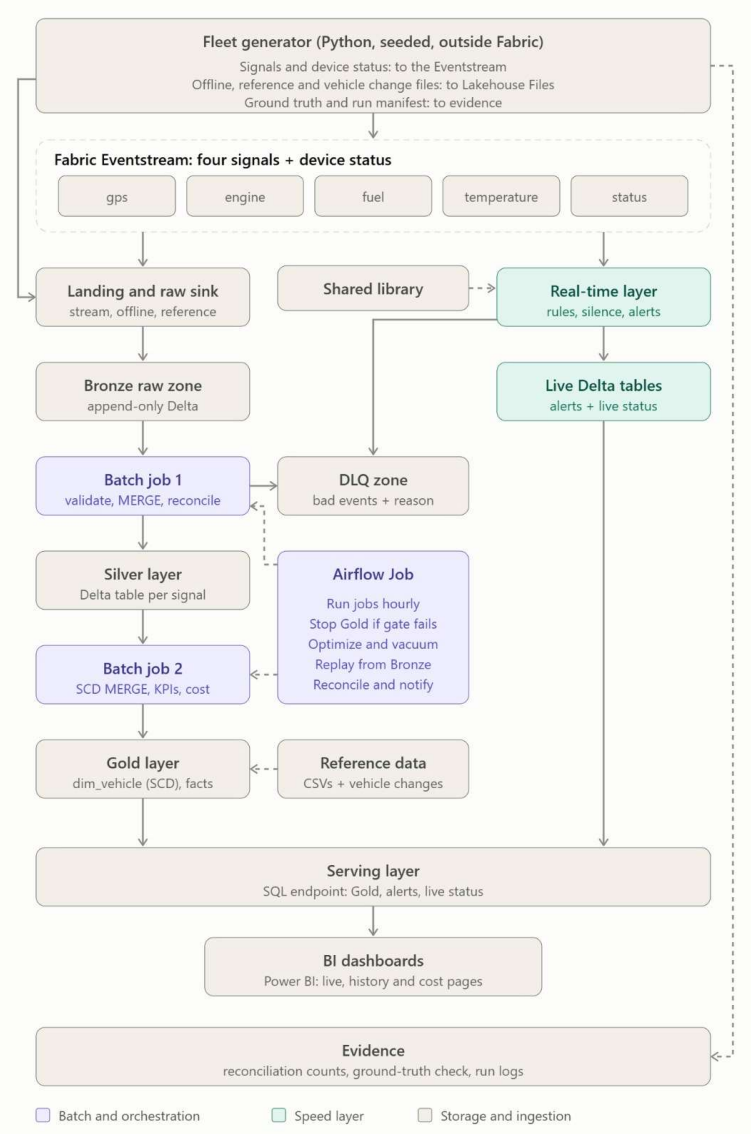


Figure 1. Architecture of the Fleet Telematics Lakehouse

- **Fleet generator:** a seeded Python generator that runs outside Fabric. It sends signals and device status to the Eventstream, writes offline, reference and vehicle change files to the Lakehouse Files area, and writes the ground truth and run manifest to the evidence folder.
- **Fabric Eventstream:** carries four signals (gps, engine, fuel, temperature) and the device status feed.

- **Landing and raw sink:** receives the stream, offline and reference data and feeds the Bronze raw zone.
- **Bronze raw zone:** append-only Delta storage of the raw messages.
- **Batch job 1:** validates messages, performs MERGE operations and reconciles the counts. Rejected messages go to the DLQ zone together with the reason.
- **Silver layer:** one Delta table per signal.
- **Batch job 2:** performs the SCD MERGE and computes KPIs and cost.
- **Gold layer:** dim_vehicle (slowly changing dimension) and the fact tables. Reference data (CSV files and vehicle changes) feeds this layer.
- **Real-time layer:** applies rules, detects silence and raises alerts, using the shared library. Its output is stored in live Delta tables (alerts and live status). Bad events from this layer are also sent to the DLQ zone.
- **Serving layer:** a SQL endpoint exposing the Gold tables, alerts and live status.
- **BI dashboards:** Power BI pages for live data, history and cost.
- **Evidence:** reconciliation counts, the ground-truth check and run logs.
- **Orchestration:** the diagram depicts an Airflow Job that runs the jobs hourly, stops Gold if the gate fails, optimizes and vacuums tables, replays from Bronze, and reconciles and notifies. The orchestration that was implemented uses a Data Factory data pipeline (Section 13).

2.2 Workspace Components

The following items exist in the Fabric workspace SIC8-Fleet-Telematics.

- **eh_fleet:** Eventhouse with a KQL database. It receives the streaming data (the bronze_*_raw tables and the alert_events table).
- **es_gps, es_engine, es_fuel, es_temperature, es_device_status:** Eventstreams that stream continuously into KQL. They are not part of the batch pipeline.
- **lh_fleet:** Lakehouse used for storage, with the schemas bronze, silver, gold, ref and ops.
- **env_fleet:** Fabric Environment that holds the custom fleet_shared library.
- **nb_p2_bronze_loader:** notebook that loads KQL data into Bronze.
- **nb_silver_setup:** notebook that prepares reference data and watermarks.
- **nb_p2_batch_job_1 and nb_p2_batch_job_1_clean:** notebooks that move data from Bronze to Silver and compute Gate 1.
- **nb_p4_fact_alerts_load:** notebook that loads KQL alerts into gold.fact_alerts.
- **nb_p3_batch_job_2:** notebook that moves data from Silver to Gold.
- **pl_fleet_batch:** Data Factory data pipeline that orchestrates the batch steps.
- **pl_rt_alert_writer:** Data Factory data pipeline that runs the KQL activity write_alerts, which appends new alerts to alert_events (Section 10.10).

- **rtd_fleet_live:** Real-Time Dashboard with two pages that reads directly from the KQL database (Sections 10.7 to 10.9).

3. Technologies and Tools

- Microsoft Fabric: workspace, Lakehouse, Eventhouse and KQL database, Eventstream, Environment, Data Factory data pipeline and notebooks.
- Delta tables, with Spark notebooks for the batch jobs (Spark SQL, PySpark and a Spark UDF for validation).
- Kusto Spark connector, used by the Bronze loader to read KQL tables with a token obtained through mssparkutils.credentials.getToken.
- Kafka protocol, used by the custom endpoint sources of the Eventstreams.
- KQL (Kusto Query Language) for the raw tables and the verification queries.
- KQL functions and update policies for real-time parsing, validation, detection and alert writing, and a Real-Time Dashboard for live monitoring.
- Python 3 for the generator and tools, with the packages pyyaml, numpy, python-dotenv and confluent-kafka.
- The fleet_shared Python library, packaged as a wheel (fleet_telematics_shared-1.0.0).
- Power BI for the dashboards.
- Git, for the project repository.
- Apache Airflow Job, which was confirmed to be available in the workspace but was not selected for orchestration.

4. Fabric Environment Setup

4.1 Workspace and Lakehouse

The workspace is SIC8-Fleet-Telematics. The trial capacity ends on approximately 23 November 2026, and the whole plan must be completed before that date.

The Lakehouse lh_fleet contains the following Files folders.

- Files/reference: reference CSV files (vehicles, drivers, routes and others).
- Files/landing/offline: bulk files of the offline windows (P1-12).
- Files/evidence: run evidence (counts, logs and screenshots).

4.2 Eventhouse and Eventstreams

One Eventstream exists per signal. Each Eventstream has one custom endpoint source (Kafka protocol) and one Eventhouse destination in Direct ingestion mode that writes the received text into a raw table of the KQL database eh_fleet. No operators are used. Figure 2 shows the Eventstream for the GPS signal, which has the same structure as those of the other signals.

- gps: Eventstream es_gps, raw table bronze_gps_raw.
- engine: Eventstream es_engine, raw table bronze_engine_raw.

- fuel: Eventstream es_fuel, raw table bronze_fuel_raw.
- temperature: Eventstream es_temperature, raw table bronze_temperature_raw.
- device_status: Eventstream es_device_status, raw table bronze_device_status_raw (empty until P1-12 is switched on).

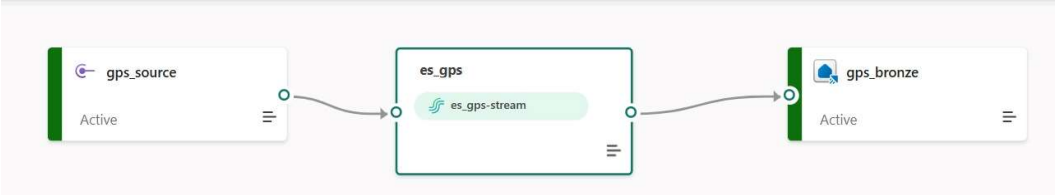


Figure 2. Eventstream es_gps: custom endpoint source, Eventstream and destination

Each raw table has exactly one column, raw_value of type string, which holds the message exactly as received, including malformed messages. The data format of the destination is TXT, because a typed format (JSON) would drop truncated messages and convert values of the wrong type, which would hide the injected faults. The tables are created before the ingestion wizard is used, with the command .create-merge table bronze_gps_raw (raw_value: string), and likewise for the other four tables. In the wizard, the existing table is selected with the format TXT. If the wizard adds a data column, the table content is inspected: if the text is in raw_value, the data column is dropped; if the text is in data, the raw_value column is dropped and the data column is renamed to raw_value. The schema of each table must show only raw_value.

4.3 Connection Settings

Connection settings are stored in a .env file with one block per signal. The values come from the Event Hub tab of each custom endpoint, on the SAS Key Authentication page. Each block contains three keys: KAFKA_<SIGNAL>_BOOTSTRAP (the host of the connection string followed by :9093), KAFKA_<SIGNAL>_TOPIC (the Event Hub name, which is the EntityPath value) and KAFKA_<SIGNAL>_CONN_STR (the primary-key connection string on one line). The same three keys are defined for GPS, engine, fuel, temperature and device status. The .env file must never be committed, keys must never be pasted into chats or notebooks, and any key that has been exposed must be regenerated.

4.4 Test Scripts

- test_all.py: sends valid test messages whose event_id starts with TEST-ALL. Options are -only <signal> and -count N. It warns when the bootstrap, topic and connection string do not match each other, or when two signals share a connection string.
- test_malformed.py: sends a valid control message (BAD-TEST-CONTROL-001) and a truncated message (BAD-TEST-001).
- load_test.py: Eventstream throughput test (Section 5.3).

5. Day-1 Tests and Design Decisions

5.1 Lakehouse Destination Test

The first test used a Lakehouse destination for the Eventstream. The findings were as follows.

- One Eventstream can carry all signals, and the signals remain separable through signal_type.
- The schema is fixed from the first message. Measurements of engine, fuel, temperature and device_status were lost.
- There is no raw text column.
- The stream offset is not available (only the partition).
- The arrival timestamp is available as EventEnqueuedUtcTime, which is the closest equivalent.
- A malformed message is silently dropped, and the original text cannot be recovered.

The conclusion was that the Lakehouse destination cannot serve as Bronze. A Spark raw-sink notebook was prepared first, but a permanently running Spark session used the trial capacity (HTTP 430, TooManyRequestsForCapacity), and the notebook was never tested with real data. It is not part of the final design.

5.2 Decision: Raw Tables in Eventhouse

Bronze consists of one raw table per signal in Eventhouse (Section 4.2), fed by direct ingestion in TXT format. Spark is used only for the batch jobs that read Bronze. The team accepted the following known limits of this choice.

- The stream offset and source_stream columns are not available in the raw table. The signal is known from the table, and offsets are not kept.
- The arrival time is not available as a column; ingestion_time() is used in KQL.
- Direct ingestion is batched, so rows can appear a few minutes late.
- The OneLake (Delta) copy of the raw tables can lag behind the KQL table, so KQL is used for exact counts.

5.3 Throughput Test

The script load_test.py uses two producer processes, 30 seconds per step, and the message mix of the evidence profile. A step passes if the delivered rate is at least 95 percent of the target, errors are zero, and the 95th percentile delivery latency is at most 2 seconds. The results were as follows.

- Target 200 messages per second: delivered 200 per second, p95 latency 0.14 s, PASS.
- Target 500: delivered 500 per second, p95 latency 0.64 s, PASS.
- Target 1,000: delivered 976 per second, p95 latency 5.59 s, FAIL (latency).
- Target 2,000: delivered 957 per second, p95 latency 15.34 s, FAIL.
- Target 3,000: delivered 1,191 per second, p95 latency 46.36 s, FAIL.

The reference rate is R = 500 messages per second, with a practical ceiling of about 1,000. The measurement was taken on the producer side, covered Eventstream ingest only, and was made before the move to Eventhouse. It should be repeated on the current setup before the evidence run.

5.4 Acceleration Factor

The acceleration factor A means that one wall-clock second equals A simulated seconds. The upper bound is given by the formula A <= (0.5 x R) / (peak active trucks x messages per truck per simulated second). With 0.085 messages per truck per second in the evidence profile, the result is A <= (0.5 x 500) / (2,500 x 0.085) = 1.18, so A = 1.

- Smoke profile (20 vehicles): proposed A = 60, maximum A with R = 500 is 52.
- Live demo profile (200 vehicles): proposed A = 30, maximum A is 5.
- Evidence profile (5,000 vehicles, 2,500 at peak): proposed A = 10, maximum A is 1.

The options for the evidence run are to lower A (six simulated hours take six real hours at A = 1), to stagger trips over 12 simulated hours (this halves the peak and is already set in config.yaml), or to relax the latency criterion (R of about 1,000 allows A of about 2.3). The value of A is not final; it remains a value in config.yaml until the team decides.

5.5 Day-1 Check Status

- One Eventstream with several endpoints keeps signals separable: confirmed.
- Raw sink keeps raw text, offsets and arrival time: raw text yes; offsets and arrival time no (Section 5.2).
- Throughput: R = 500 messages per second, ceiling about 1,000 (to be repeated).
- Malformed message kept in the raw table: see Section 14.4, check 2.
- Power BI refresh and Direct Lake lag: not tested yet.
- Airflow Job: confirmed available in the workspace.

6. Data Generator and Reference Data

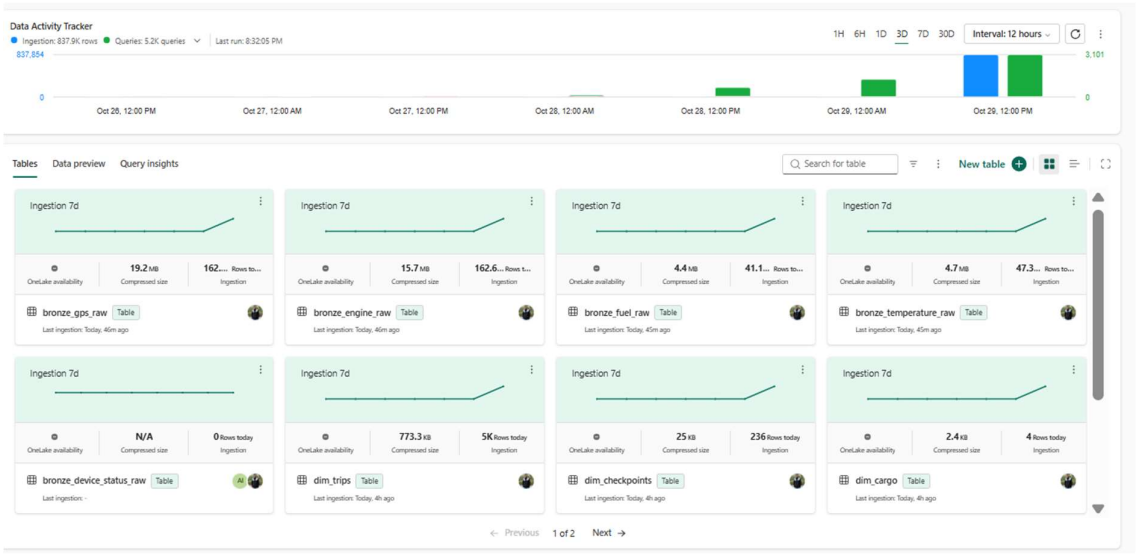
6.1 Project Files

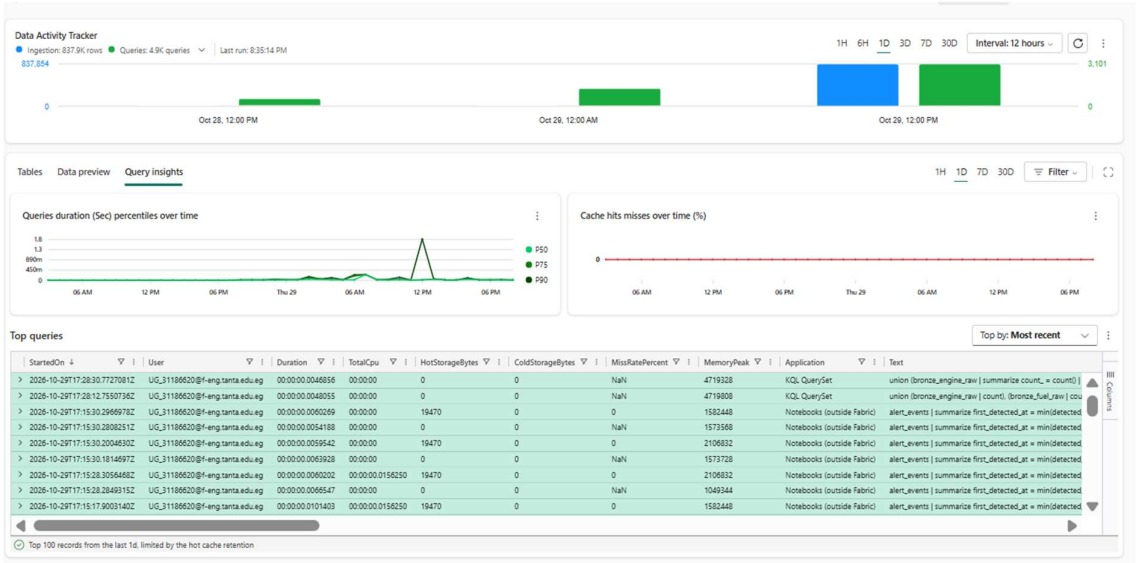
- config.yaml: single configuration file containing the seed, mode, intervals, acceleration factor, and anomaly, fault and offline settings.
- generate_reference_data.py: generates the reference CSV files. The mode is read from config.yaml only. With reference_file_faults set to true it also writes reference_faults/.
- generate_stream.py: truck simulation (generator v1.3). It sends messages to the Eventstream endpoints and writes the evidence files.
- verify_stream.py: checks stream_messages.jsonl and the bulk files against the ground truth. It requires --sink jsonl.
- audit_phase1.py: checks the project folder against the Phase 1 plan.
- config.py and rng.py: configuration loader and seeded random helper.

- test_all.py, test_malformed.py and load_test.py: Eventstream tests.
- fleet_telematics_shared-1.0.0-py3-none-any.whl: the shared library, imported as fleet_shared.

The file config.yaml is replaced as a whole only when a new feature adds keys; otherwise single lines (mainly the mode) are edited.

6.2 Run Modes and Configuration





A run mode selects the fleet size, the number of routes, the message intervals and the acceleration factor. The intervals are given in seconds for gps, engine, fuel and temperature respectively.

- **smoke:** 20 vehicles, 3 routes, intervals 10 / 10 / 60 / 30, A = 50. End-to-end check; it is the current mode.

- **live_demo:** 200 vehicles, 8 routes, intervals 10 / 10 / 60 / 30, A = 5. Used to show alerts live.
- **evidence:** 5,000 vehicles, 12 routes, intervals 30 / 30 / 120 / 60, A = 1. Scale proof of about 5.4 million messages; it is done last (Section 15.2).
- **clean:** 20 vehicles, 3 routes, intervals 10 / 10 / 60 / 30, A = 50. Baseline with no anomalies, no faults, no late trips and no offline windows.

The parameter simulation.days (default 1) gives one trip per vehicle per day. At A = 50, a simulated day takes about 30 minutes. The main configuration keys are listed below.

- seed: the seed of every random draw.
- mode, acceleration_factor and profiles.<mode>.*: the active profile and its fleet size, routes, intervals, A and zero_rates.
- simulation.start, departure_window_start_hour and days: the simulated start, the first departure hour and the number of days.
- refrigerated_share, driver_ratio and fuel_prices: fleet composition and declared fuel price.
- scd.*: the hour of the scripted vehicle changes and the bulk change file; bulk counts per 5,000 vehicles.
- injection.min_per_type: the smallest number of each injected case in a run (ignored in clean mode).
- rates.cold_chain_breach, idling and route_deviation: shares of trips. One third of the breaches are short blips, and a GPS wander is added at the deviation rate.
- rates.late_arrival and late_within_grace: shares of trips (Section 6.7).
- Data fault rates, unit_error_vehicle_share, sensor_fault_trips, stuck_sensor_trips and poison_batch_mode (Section 6.6), and poison_batch.* (malformed rate, start hour and length of the poison batch).
- reference_file_faults: when true, also writes reference_faults/.
- offline.*: offline window settings (Section 6.8).
- business_rules: thresholds written to reference/business_rules.csv.

6.3 Reference Data

The reference data consists of routes, route checkpoints (15 to 25 per route), cargo types, fuel prices, vehicles_initial.csv, drivers, trips and business_rules.csv, all produced from the same configuration. The smoke size is 20 vehicles, 13 drivers, 3 routes, 59 checkpoints, 4 cargo types, 21 trips and 32 rules. Trips do not overlap for a vehicle or a driver, and cold cargo is assigned only to refrigeration-capable vehicle versions.

Figure 3 shows the output of the reference data generator in evidence mode (5,000 vehicles, 12 routes, seed 42).

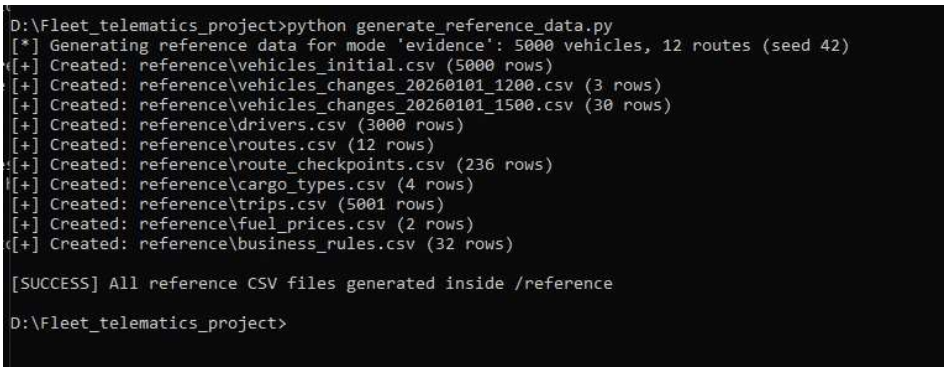


Figure 3. Output of generate_reference_data.py in evidence mode

The evidence-mode output consists of vehicles_initial.csv (5,000 rows), two vehicle change files (3 rows and 30 rows), drivers.csv (3,000 rows), routes.csv (12 rows), route_checkpoints.csv (236 rows), cargo_types.csv (4 rows), trips.csv (5,001 rows), fuel_prices.csv (2 rows) and business_rules.csv (32 rows).

6.4 Simulation and Ground Truth

The simulation uses one state machine per truck on a simulated clock. Sequence numbers are maintained per device and signal, and messages are keyed by vehicle_id. The plan of injected events is written to evidence/injected_events_log.csv and evidence/run_manifest.json before anything is sent. The ground-truth types are listed below; the details field holds negative_case and expected_alert.

- COLD_CHAIN_BREACH (alert expected): temperature above the cargo maximum for longer than the alert hold.
- COLD_CHAIN_BLIP (no alert): above the maximum, but clearly shorter than the alert hold.
- EXCESSIVE_IDLING (alert expected): engine on and truck stopped for 15 to 60 minutes.
- SHORT_STOP (no alert): a normal stop of 1 to 4 minutes.
- ROUTE_DEVIATION (alert expected): GPS 1 to 5 km off the route for 5 to 20 minutes.
- GPS_WANDER (no alert): a wander of 100 to 350 m, inside the allowed deviation.
- LATE_ARRIVAL and LATE_WITHIN_GRACE: see Section 6.7.
- OFFLINE_WINDOW and BUFFER_OVERFLOW: see Section 6.8.
- Data faults (SENSOR_FAULT, STUCK_SENSOR, UNIT_ERROR, POISON_BATCH and others): see Section 6.6.

6.5 Vehicle Changes (Slowly Changing Dimension)

Vehicle changes are delivered as files named vehicles_changes_YYYYMMDD_HHMM.csv. The scripted changes are a retrofit of vehicle V002, a registration correction of V007 and a device swap of V010. Bulk changes appear in the larger profiles.

6.6 Data-Fault Injection (P1-10)

Faults are applied to single messages after they are built, so that the simulated world is correct while the data is wrong. Sensor faults and unit errors are planned per trip. Every fault is written to injected_events_log.csv before anything is sent, together with the event_id of the affected message and details (device_id, sequence_number, handling, expected_error_code, and the expected field values, where "__missing__" denotes a removed field).

Reproducibility is preserved as follows. Data faults use their own random stream per vehicle, so the physical anomalies are identical with or without faults. A dry pass collects the faults before sending. At most one fault is applied per message. The clean mode injects nothing. The injection types, default rates and expected handling are listed below.

- DROPPED_MESSAGE (0.3 percent, all four signals): the message is built and numbered but never sent, producing a sequence gap. Expected handling: gap and completeness checks, DATA_GAP.
- DUPLICATE (0.5 percent, all four signals): the same message is sent twice. It is removed in Silver by event_id.
- NEAR_DUPLICATE (0.2 percent, all four signals): same vehicle, signal, time and sequence number with a new event_id. Handled by the composite-key rule.
- MALFORMED (0.2 percent, all four signals): the JSON is cut short. DLQ, PARSE_ERROR.
- MISSING_FIELD (0.2 percent, all four signals): vehicle_id, event_time or trip_id is removed. DLQ, MISSING_REQUIRED_FIELD.
- WRONG_TYPE (0.2 percent, all four signals): one numeric field is sent as text. DLQ, TYPE_MISMATCH.
- INVALID_GPS (0.1 percent, gps): coordinates (0,0) or a latitude above 90. DLQ, INVALID_GPS.
- IMPOSSIBLE_VALUE (0.1 percent, gps and engine): speed of 400 km/h or negative RPM. DLQ, OUT_OF_RANGE.
- BAD_TIMESTAMP (0.1 percent, all four signals): a future date, the format dd/mm/yyyy, or no time zone. DLQ, INVALID_TIMESTAMP.
- UNKNOWN_VEHICLE (0.05 percent, all four signals): vehicle_id of the form VUNKnnnn. DLQ, UNKNOWN_VEHICLE.
- SCHEMA_DRIFT (0.05 percent, all four signals): half of the cases add the field firmware_version and half use schema_version 2. The extra field is kept and flagged; version 2 goes to the DLQ.
- TEMPERATURE_SPIKE (0.1 percent, temperature): one reading is raised by 10 to 15 degrees Celsius. It is kept and flagged SPIKE.
- UNIT_ERROR (1 percent of refrigerated vehicles, temperature): all cold trips report Fahrenheit (temperature_unit = "F"). The values are converted to Celsius and flagged UNIT_CONVERTED.

- SENSOR_FAULT (2 percent of cold trips, temperature): 5 to 30 minutes of null readings with sensor_status FAULT. Kept and flagged SENSOR_NULL; an alert is expected.
- STUCK_SENSOR (0.5 percent of cold trips, temperature): a frozen reading longer than flatline_readings. Kept and flagged FLATLINE; no alert is expected.
- POISON_BATCH (off by default, all signals): malformed rate of 10 percent for 5 simulated minutes on day 1. A HIGH_INVALID_RATE alert is expected and Gate 1 stops Gold.

About 1 percent of messages are invalid, which is below the 2 percent alert threshold and the 5 percent gate limit. The parameter injection.min_per_type guarantees at least one sensor fault, one stuck sensor and one unit-error vehicle in the smoke run; per-message faults are random.

6.7 Late-Arrival Injection (P1-11)

- Slow trip (rates.late_arrival, 5 percent of trips): the truck arrives 10 to 60 minutes after the planned end plus grace. A LATE_ARRIVAL alert with cause INJECTED_SLOW_TRIP is expected.
- Near-miss (rates.late_within_grace, 5 percent of trips): the truck arrives after the planned end but inside the grace period. No alert is expected (LATE_WITHIN_GRACE).

The truck paces itself to a chosen arrival time instead of using a fixed slowdown, because a fixed speed of 60 to 80 percent cannot reach a window of 10 to 60 minutes on routes of very different length. No extra stops are added. The ground truth is derived from the real arrival times of the dry pass: every trip that arrives later than the planned end plus grace is logged as LATE_ARRIVAL, and trips that arrive late as a side effect of idling receive the cause SIDE_EFFECT. A late trip must end before the next trip of the same vehicle starts, and trips without room are skipped. Drivers are not checked, because no message carries a driver. The feature uses its own random stream, so the other anomalies and faults are unchanged.

6.8 Offline Windows (P1-12)

Offline windows (store-and-forward) are implemented in generator v1.3 and are off by default (offline.enabled: false). With the feature off, the output is identical to that of a run without P1-12.

While offline, the device keeps measuring and stores each reading with its true consecutive sequence number. On reconnection it sends a device_status report (stored_* counters, dropped_messages, upload_method, RECONNECTED, event_time, offline_from, reconnected_at and upload_id), followed by the stored readings, either as a burst on the stream (STREAM_BURST) or as a bulk file (BULK_FILE). The rules are as follows.

- Share of trips: 8 percent (offline.trips_share).
- Outage length when the buffer survives: 3 to 60 minutes (min_minutes, max_minutes), cut to 95 percent of the buffer if shorter.
- Buffer survives or overflows: 85 percent and 15 percent (buffer_survives_share).

- Buffer capacity: 300 messages per signal (buffer_capacity_messages); the oldest messages are lost on overflow.
- Overflow window: (capacity x GPS interval) plus 5 to 20 minutes (overflow_extra_min and overflow_extra_max_minutes). A window must be long enough to overflow; with 3 to 60 minute windows and 30 second intervals, the evidence profile stores only 120 messages per hour per signal, so a buffer of 300 could never overflow otherwise.
- Burst or file: the average burst share is 0.5 (burst_share), and longer outages favour the file.

Bulk files are written to <out>/landing/offline/<vehicle_id>_<upload_id>.jsonl (for example V009_UPL00001.jsonl), with one JSON message per line and the same schemas as the stream. They are uploaded to Files/landing/offline/ and are not part of stream_messages.jsonl. The generator removes old *.jsonl files in that folder at the start of a run.

The ground truth contains OFFLINE_WINDOW entries (start and end of the outage, built_by_signal, the event_id of the report, upload_method, stored_by_signal, dropped_total, upload_id, file_name, expected_episode_status CLOSED or CLOSED_WITH_LOSS, and the expected alert DEVICE_OFFLINE). When readings were lost, a BUFFER_OVERFLOW entry is added (lost_ranges with device, signal, first and last sequence number, and the expected alert DATA_GAP).

The feature has the following limits. No data faults (P1-10) are applied to stored readings, because a dropped message would otherwise look like a loss in transit. Offline windows are placed where no other planned anomaly exists, so they never hide a breach or a deviation. The device_status endpoint must be defined in .env, and the generator stops with a message if it is missing. Loading the bulk files into Bronze (file ingestion, ops_ingestion_log) is a pipeline task, not a generator task.

6.9 Reference-File Faults

When reference_file_faults is true, generate_reference_data.py also writes reference_faults/, while the normal reference/ folder stays valid. The faulty files are uploaded to a separate folder (for example Files/reference_faults), never to Files/reference, and are intended for testing the file-ingestion step (DLQ stage FILE_INGEST). The flag is set back to false after generation. The faults are as follows.

- trips.csv: an extra trip overlapping a trip of the same vehicle, and an extra cold-cargo trip on a vehicle that is never refrigeration-capable.
- cargo_types.csv: a cargo BAD_RANGE with a minimum of 10 above a maximum of 5.
- routes.csv: a row R999 with an extra delimiter.
- drivers.csv: the column license_category removed.
- reference_faults_log.csv: the ground truth of these faults.

6.10 Generator Outputs

- reference/*.csv: reference data and vehicles_changes_*.csv.
- evidence/run_manifest.json: run_id, seed, profile, vehicle_count, sim_start, wall_start, acceleration_factor, intervals, rates, offline settings and planned injections.
- evidence/injected_events_log.csv: the ground truth, written before sending and read only by the evaluation step.
- evidence/trip_outcomes.csv: actual start and end, distance, fuel, idle minutes and minutes late beyond grace.
- evidence/run_summary.json: messages sent per signal, dropped, built, offline counters and delivery result.
- evidence/stream_messages.jsonl: all messages (only with --sink jsonl).
- evidence/landing/offline/*.jsonl: bulk files of the offline windows.

7. Shared Library fleet_shared

The shared library is distributed as the wheel fleet_telematics_shared-1.0.0-py3-none-any.whl, is imported as fleet_shared, and has no dependencies. It is installed in the Fabric Environment env_fleet. The modules are as follows.

- schema: schema version 1 for the five signals.
- rules: default rules (identical to business_rules.csv) and a loader.
- validation: validate_message() with the error codes PARSE_ERROR, MISSING_REQUIRED_FIELD, TYPE_MISMATCH, OUT_OF_RANGE, INVALID_GPS, INVALID_TIMESTAMP, UNKNOWN_VEHICLE and UNKNOWN_TRIP; Fahrenheit to Celsius conversion; flags such as SENSOR_NULL; and spike, flatline and GPS-jump helpers.
- dedup: the event_id key, the near-duplicate key and the winning-copy rule.
- geo: Haversine distance and distance from the route path.

Local tests report 22 passed and 1 skipped (PySpark). A notebook attached to env_fleet printed the version 1.0.0 and PARSE_ERROR. A check of a message file can be run with python -m fleet_shared.selfcheck evidence/stream_messages.jsonl reference/business_rules.csv.

8. Bronze Layer

Bronze consists of the raw KQL tables in eh_fleet (Section 4.2) and their copies in the Lakehouse. The notebook nb_p2_bronze_loader performs the transfer with the following behaviour.

- It reads the KQL tables bronze_{signal}_raw for the five signals (gps, engine, fuel, temperature, device_status) through the Kusto Spark connector, using a token from mssparkutils.credentials.getToken.

- The load is incremental. It is based on ingestion_time() and the watermark table ops.ops_bronze_load_watermark. The upper bound is the current time minus 2 minutes (SAFETY_LAG_MIN).
- It writes to bronze.bronze_{signal}_raw as Delta tables partitioned by ingestion_date.
- A retry guard deletes the window of the run before rewriting it, which makes retries safe.
- The first load contained gps 30,895, engine 30,926, fuel 5,176, temperature 7,197 and device_status 0 rows.

The notebook nb_silver_setup prepares the schemas and the ref.* tables from the files under Files/reference (overwritten on every run) and initializes ops.ops_watermark if it is empty. It is safe to rerun. If the reference files are static, it can be removed from the schedule after the first run.

9. Silver Layer

9.1 Purpose and Interfaces

The notebook nb_p2_batch_job_1 is the batch job that turns the raw telemetry in Bronze into clean, typed and enriched Silver tables. It loops over four signals: gps, engine, fuel and temperature. For every signal it reads only the new Bronze rows (the watermark window), validates each message with the fleet_shared library, sends rejected messages to the DLQ, enriches the valid ones, removes duplicates, adds quality flags, merges the result into silver.silver_<signal>, records the counts in an ingestion log and moves the watermark. When all signals are done, Gate 1 computes the share of invalid rows and returns a JSON status as the notebook exit value.

The inputs are as follows.

- bronze.bronze_<signal>_raw: the raw message text (raw_value) with ingestion_time, source_type, source_file and file_line.
- ref.ref_vehicles_raw: vehicle versions with active_from, active_to, updated_at and fuel_tank_capacity_liters.
- ref.ref_trips: the trip reference (trip_id, route_id, driver_id, cargo_type_id).
- Files/evidence/run_manifest.json and run_summary.json: run timing (sim_start, wall_start, acceleration factor) and the message counts of the run.
- fleet_shared 1.0.0 from the environment env_fleet: message validation, field definitions, error codes and flag names.

The outputs are as follows.

- silver.silver_gps, silver_engine, silver_fuel and silver_temperature: validated, typed and enriched rows in Delta tables partitioned by event_date.
- ops.dlq_events: rejected messages with the error code (stage BATCH).
- ops.ops_ingestion_log: one row per signal per run, plus one Gate 1 row per run.
- ops.ops_watermark: the last processed Bronze ingestion time per signal.

- The notebook exit value: the Gate 1 JSON status.

9.2 Processing Flow

The function process_signal(signal) performs the following steps in order. The function main() calls it for each signal and then runs Gate 1.

1. Select the window of new rows (get_window): the lower bound is the saved watermark and the upper bound is the newest Bronze ingestion_time.
2. Validate every row (load_lookups, make_validate_udf, validate), producing good rows and bad rows.
3. Check that the vehicle is active on the event date (split_by_vehicle_activity); inactive-vehicle rows move to the bad rows.
4. Write the bad rows to the DLQ (write_dlq), into ops.dlq_events.
5. Parse, enrich and compute lateness (parse_and_enrich): typed columns, dq_flags text, route, driver and cargo, and lateness_sec.
6. Remove duplicates (dedup_batch): one row per event_id, with the duplicates counted.
7. Add sequence quality flags (add_sequence_flags): SPIKE, FLATLINE and IMPOSSIBLE_JUMP are appended to dq_flags.
8. MERGE into Silver (merge_silver): only event_ids not yet in Silver are inserted.
9. Log the counts (write_log): one row in ops.ops_ingestion_log.
10. Move the watermark (set_watermark): the watermark is set to the upper bound and is always the last step.
11. Run Gate 1 (run_gate1, write_gate_log, main): the JSON status is stored in the log and returned as the exit value.

9.3 Configuration of the Job

The session time zone is set to UTC so that timestamps match Kusto and Bronze. The watermark text keeps microseconds so that no Bronze row is reloaded. The notebook function load_manifest() reads Files/evidence/run_manifest.json and keeps sim_start and wall_start (as epoch seconds), the acceleration factor A and fast_mode. The function check_manifest_against_bronze() compares per_signal in run_summary.json with the Bronze row counts, prints OK or MISMATCH per signal, and prints a warning if the manifest does not belong to the data or if the run used fast mode. The job settings are as follows.

- ENABLE_ACTIVE_CHECK = True: reject rows whose vehicle is not active on the event date.
- SPIKE_DELTA_C = 8.0: a temperature change from the previous reading above this value gives SPIKE.
- FLATLINE_N = 10: this many identical consecutive temperature readings give FLATLINE.
- MAX_JUMP_SPEED_KMH = 200.0: an implied GPS speed between two consecutive points above this value gives IMPOSSIBLE_JUMP.

- GATE_MAX_INVALID_PCT = 5.0: Gate 1 fails above this share of invalid rows (quality_gate_max_invalid_pct).

9.4 Validation and Reference Lookups

The function load_lookups() builds three small reference structures that are shipped to the executors inside the validation UDF: known_vehicles (the set of vehicle ids in ref_vehicles_raw), known_trips (the set of trip ids in ref_trips), and tank_cap (one fuel tank capacity per vehicle, taken from the latest row by updated_at). A fuel level above the capacity is rejected as OUT_OF_RANGE.

The function make_validate_udf() wraps fleet_shared.validate_message() in a Spark UDF that returns the struct (ok, error_code, error_message, retryable, flags, record_json). The library also receives the simulated time "now" for the future-timestamp check. The function simulated_now(upper) computes it from the manifest as sim_start plus (upper minus wall_start) multiplied by A, as a naive datetime. The function validate() reads the window from Bronze, applies the UDF, caches the result, counts the Bronze rows and splits them into good rows (ok is true) and bad rows (ok is false). Unknown vehicles and trips are detected by the lookups (UNKNOWN_VEHICLE, UNKNOWN_TRIP).

9.5 Vehicle Activity Check

After library validation, good rows pass through split_by_vehicle_activity() (controlled by ENABLE_ACTIVE_CHECK). A row is kept only if at least one version of its vehicle in ref_vehicles_raw is active on the event date, that is, active_from is on or before the event date and the event date is on or before active_to, where an empty or NULL active_to means that the vehicle is still active. The check is a broadcast semi join (active rows) and anti join (inactive rows) on the vehicle id and the date range. Inactive rows are added to the bad rows with the error code UNKNOWN_VEHICLE and the message "Vehicle not active at event time".

9.6 Dead-Letter Queue

Rejected rows are written to ops.dlq_events with stage BATCH, the error code and message, the event_id (read with get_json_object, which returns NULL instead of failing on broken JSON), the raw text and the Bronze arrival time. The write is idempotent: before inserting, the function deletes the DLQ rows of the same signal, stage and Bronze time window, so a re-run never doubles them.

9.7 Parsing, Enrichment and Lateness

Good rows carry the cleaned record as JSON text. The function parse_and_enrich builds a DDL schema from the library field list (fs.SIGNAL_FIELDS[signal]) plus the common fields, so that the same code works for every signal, and parses the JSON with from_json. It stores dq_flags as comma-separated text (for example UNIT_CONVERTED,SPIKE), or NULL when no flag is present, converts event_time to a timestamp, derives event_date (the partition column), joins ref_trips on trip_id (broadcast) to add route_id, driver_id and cargo_type_id, and computes lateness_sec.

Because event_time is simulated time and ingestion_time is wall-clock time, they cannot be subtracted directly. The simulated time is first mapped to the wall-clock time at which the message was expected to arrive, using the values of the run manifest, and the expected arrival is then subtracted from the real ingestion time:

expected_arrival = wall_start + (event_time - sim_start) / A

lateness_sec = ingestion_time - expected_arrival (in seconds, rounded to 3 decimals)

9.8 Deduplication

The function dedup_batch keeps one row per event_id (the library DEDUP_KEY). A window partitioned by event_id and ordered by ingestion_time ascending keeps the first copy that arrived. The function returns the deduplicated DataFrame, the number of rows kept and the number of duplicates removed; the removed count is written to the ingestion log.

9.9 Sequence Quality Flags

Some flags need the previous reading of the same vehicle. They are computed on the deduplicated batch with window functions and appended to dq_flags with concat_ws, so that the column remains a single text value. The previous row is looked up inside the current batch. Engine and fuel have no sequence flags.

- SPIKE (temperature): the absolute difference to the previous reading of the same vehicle and sensor (ordered by event_time and sequence_number) is above SPIKE_DELTA_C.
- FLATLINE (temperature): the last FLATLINE_N readings of the same vehicle and sensor are all present and identical.
- IMPOSSIBLE_JUMP (gps): the Haversine distance to the previous point divided by the time gap is above MAX_JUMP_SPEED_KMH; the time gap must be positive.

9.10 MERGE, Logging and Watermark

The function merge_silver selects the deduplicated rows in the column order of the target table, registers them as a temporary view and merges them into silver.silver_<signal> on event_id with a WHEN NOT MATCHED THEN INSERT clause. Only event_ids that are not yet in Silver are inserted, so repeating a run inserts nothing. The function write_log appends one row per signal per run to ops.ops_ingestion_log, with the window, the Bronze, Silver, DLQ and duplicate counts, and the status (SUCCESS or FAILED with the error text). The function set_watermark updates ops.ops_watermark to the upper bound and is always the last step of a successful run.

If any step raises an exception, a FAILED row is logged, the exception is re-raised, and the watermark is not moved, so the same window is processed again on the next run. The cached DataFrame is always un-persisted in the finally block. At the end, process_signal prints the balance check (Bronze = Silver + DLQ + duplicates removed) and returns the four counts.

9.11 Gate 1

Gate 1 (run_gate1) computes the invalid share as DLQ rows divided by Bronze rows, overall and per signal over the signals processed in the run. The status is FAIL when the overall share or the share of any signal is above GATE_MAX_INVALID_PCT (5 percent), and PASS otherwise. Silver is written in both cases; the gate only tells the caller whether to continue. If no signal had new data, the status is PASS with the note "no new data".

The same JSON is stored in ops.ops_ingestion_log with stage GATE_1 and signal ALL (the JSON in error_message and PASS or FAIL in status). The count columns are left NULL on purpose, so that sums over stage BATCH are not counted twice. The JSON contains the fields status, invalid_pct (overall invalid share in percent), threshold_pct (5.0), per_signal (invalid share per processed signal), failed_signals (signals above the limit, an empty list when none), and the totals rows_bronze and rows_dlq. The last cell of the notebook returns this JSON with mssparkutils.notebook.exit(json.dumps(gate)), so that a pipeline can read it and decide whether the following jobs run.

9.12 Verification Cells

The notebook contains queries used to check each run: event_id uniqueness, NULL keys and NULL event times in a Silver table; DLQ counts per signal and error code; enrichment completeness (NULL route, driver or cargo, trip ids missing from ref_trips, and trip_id uniqueness in ref_trips); lateness_sec statistics per Silver table; dq_flags value counts for gps and temperature; and an isolated test that sends a message with the unknown vehicle V099 through validate_message and expects UNKNOWN_VEHICLE (it writes nothing).

9.13 Silver Tables

The four Silver tables silver.silver_gps, silver_engine, silver_fuel and silver_temperature are Delta tables partitioned by event_date. Every table has the same 14 common columns followed by the columns of its signal. The key is event_id.

Common columns (all four tables):

- event_id (STRING): unique id of the message; primary key, MERGE key and duplicate key.
- vehicle_id (STRING): vehicle that sent the message, checked against ref_vehicles_raw.
- trip_id (STRING): trip the vehicle was on; join key to ref_trips.
- device_id (STRING): telematics device that sent the message (for example DEV_V001).
- event_time (TIMESTAMP): when the reading was taken (simulated time, UTC).
- event_date (DATE): date part of event_time; partition column.
- sequence_number (BIGINT): per-device message counter that orders readings and reveals gaps.
- schema_version (INT): version of the message schema (1).

- ingestion_time (TIMESTAMP): when the row was ingested into Bronze (wall-clock, UTC); basis of the watermark and of lateness_sec.
- lateness_sec (DOUBLE): seconds between the expected arrival (from the run manifest) and the real ingestion time.
- dq_flags (STRING): comma-separated data-quality flags; NULL when no flag is present.
- route_id (STRING): route of the trip (from ref_trips).
- driver_id (STRING): driver of the trip (from ref_trips).
- cargo_type_id (STRING): cargo type of the trip, for example DRY_GOODS, MEDICINE_COLD, FROZEN_FOOD or FOOD_CHILLED (from ref_trips).

silver_gps (19 columns: 14 common and 5 specific):

- latitude (DOUBLE): latitude in decimal degrees.
- longitude (DOUBLE): longitude in decimal degrees.
- speed_kmh (DOUBLE): ground speed reported by the GPS, in km/h.
- heading_degrees (DOUBLE): direction of travel in degrees.
- gps_accuracy_meters (DOUBLE): reported position accuracy in meters (larger is worse).

silver_engine (19 columns: 14 common and 5 specific):

- engine_on (BOOLEAN): whether the engine is running.
- engine_rpm (INT): engine speed in revolutions per minute.
- engine_temperature_c (DOUBLE): engine temperature in degrees Celsius.
- vehicle_speed_kmh (DOUBLE): vehicle speed from the engine unit, in km/h.
- battery_voltage_v (DOUBLE): battery voltage in volts.

silver_fuel (18 columns: 14 common and 4 specific):

- fuel_level_liters (DOUBLE): fuel in the tank, checked against the vehicle tank capacity.
- trip_fuel_used_liters (DOUBLE): fuel used during the trip.
- odometer_km (DOUBLE): odometer reading in km.
- is_refueling (BOOLEAN): whether the vehicle is refueling at this reading.

silver_temperature (20 columns: 14 common and 6 specific):

- shipment_id (STRING): shipment carried by the vehicle.
- sensor_id (STRING): temperature sensor that produced the reading.
- temperature_value (DOUBLE): measured temperature in degrees Celsius (Fahrenheit readings are converted and flagged UNIT_CONVERTED).
- temperature_unit (STRING): unit of temperature_value after validation (C).
- setpoint_temperature_c (DOUBLE): target temperature of the cold chain in degrees Celsius.

- sensor_status (STRING): sensor health reported by the device (for example OK).

9.14 Supporting Tables

ops.dlq_events contains the following columns: dlq_id (unique uuid), batch_id (links to ops_ingestion_log), stage (BATCH for this notebook), signal, error_code, error_message, event_id (read from the raw text when possible, otherwise NULL), raw_value (the exact raw message text, kept for replay and analysis), source_type, source_file and file_line (origin of the Bronze row), bronze_ingestion_time (used for the window and for idempotent deletes), is_retryable (whether the error can disappear later, for example when a vehicle is added to the reference data), and created_at.

ops.ops_ingestion_log contains log_id, batch_id (NULL on the Gate 1 row), job_name (nb_p2_batch_job_1), signal (or ALL on the Gate 1 row), stage (BATCH or GATE_1), source_file (file ingestion only; NULL here), window_start_utc and window_end_utc (ISO text), rows_bronze, rows_silver, rows_dlq, rows_duplicates_removed, status (SUCCESS or FAILED for BATCH rows; PASS or FAIL for the GATE_1 row), error_message (error text for FAILED rows; the Gate 1 JSON for the GATE_1 row), started_at and finished_at.

ops.ops_watermark contains one row per signal (gps, engine, fuel, temperature, device_status) with watermark_utc (the upper bound of the last successful run as ISO text with microseconds, where 1900-01-01T00:00:00Z means that nothing has been processed yet) and updated_at.

9.15 Data-Quality Reference

The dq_flags values are as follows. Several flags on one row are stored in one text value separated by commas.

- UNIT_CONVERTED: temperature received in Fahrenheit and converted to Celsius (fleet_shared).
- SENSOR_NULL: the sensor reported no value (fleet_shared).
- SENSOR_FAULT: the sensor status reports a fault (fleet_shared).
- LOW_ACCURACY: GPS accuracy is poor (fleet_shared).
- SCHEMA_DRIFT: the message differs from the expected schema but is still usable (fleet_shared).
- SPIKE: temperature jumped from the previous reading by more than the configured limit (notebook).
- FLATLINE: the temperature sensor repeated the same value for the configured number of readings (notebook).
- IMPOSSIBLE_JUMP: the GPS position moved faster than the configured speed limit (notebook).

The DLQ error codes are as follows.

- PARSE_ERROR: the raw text is not valid JSON.
- MISSING_REQUIRED_FIELD: a required field of the signal is missing.
- TYPE_MISMATCH: a field has the wrong data type.

- OUT_OF_RANGE: a value is outside the allowed range, including a fuel level above the tank capacity.
- INVALID_GPS: latitude or longitude is not a valid position.
- INVALID_TIMESTAMP: the event time cannot be parsed or is in the future compared with the simulated time.
- UNKNOWN_VEHICLE: the vehicle is not in the reference data, or is not active on the event date.
- UNKNOWN_TRIP: the trip is not in the reference data.
- UNKNOWN_SIGNAL_TYPE: the signal type is not supported.
- SCHEMA_DRIFT: the schema version is not supported.

9.16 Re-running and Rebuilding

A re-run is safe for the following reasons. A signal whose window is empty prints "nothing to do" and writes nothing, and the watermark moves only after a successful run. The DLQ rows of the same signal and window are deleted before new ones are written. The MERGE inserts only event_ids that are not in Silver. After a failure, a FAILED row is logged and the watermark is unchanged, so the same window is processed again.

Silver is derived completely from Bronze, which is never modified. To rebuild Silver, the Silver tables are emptied, the batch log and DLQ rows are cleared, and the watermarks are reset; the function cells and main() are then run again. These statements are run in a cell of their own. Emptying Silver without resetting the watermarks leaves Silver empty, because the job then finds no new Bronze rows, so both actions are always performed together.

10. Real-Time Processing and Dashboard

This section describes the real-time analysis pipeline that transforms incoming fleet telemetry into validated events and operational alerts. It also describes the live dashboard that presents the alerts and the health of the pipeline, and the scheduler that persists alerts automatically. The section focuses on the processing flow rather than on implementation-level code. The processing is performed in the KQL database eh_fleet, which receives the streaming data (Section 4.2).

10.1 Process Overview

The real-time pipeline consists of the following seven stages.

1. **Reference data:** loads fleet, route, trip, cargo, checkpoint, vehicle and rule metadata. Output: KQL dimension tables and route geometry.
2. **Parse and validate:** parses raw telemetry, checks schema, data types, timestamps and ranges, and validates references. Output: the validated rt_* tables and the dead-letter table dlq_speed.
3. **Detection:** evaluates telemetry against operational conditions and persistence thresholds. Output: cold-chain, idling and route-deviation candidates.

4. **Alert writing:** standardizes alert records and appends newly identified alerts. Output: the table alert_events.
5. **Evaluation:** compares rejected records and detected alerts with the test ground truth. Output: parity, precision and recall results.
6. **Live dashboard:** presents operational alerts and pipeline health in the Real-Time Dashboard rtd_fleet_live (Sections 10.7 to 10.9). Output: two dashboard pages.
7. **Scheduler:** runs the alert writer automatically on a defined schedule. Output: scheduled updates to alert_events.

10.2 Reference Tables in KQL

The pipeline begins by preparing reference data in the KQL database. These tables describe the business context needed to interpret telemetry: trips, routes, checkpoints, cargo types, operational rules and vehicles. The route-line dataset is derived from checkpoint coordinates ordered by route and sequence, which creates a line geometry that is used to measure how far a vehicle is from its planned route.

Reference data is used throughout validation and detection. Trip and vehicle records help identify unknown entities, route settings provide deviation thresholds, and cargo settings define acceptable temperature ranges and alert hold times.

10.3 Parse and Validation Layer (rt_* and dlq_speed)

Raw GPS, engine, fuel and temperature events are read from their Bronze source tables. Dedicated KQL functions parse each JSON payload and apply common checks before a record is accepted. The validation includes the following checks.

- Payload parsing and schema-version checks.
- Required identifiers and signal-type consistency.
- Data-type checks and signal-specific range checks.
- Timestamp format and plausibility checks.
- Reference validation against known vehicles and trips.
- Temperature conversion from Fahrenheit to Celsius when required, with the data-quality flags retained.

Records that pass validation are projected into the corresponding typed real-time table: rt_gps, rt_engine, rt_fuel or rt_temperature. Rejected records are written to dlq_speed together with the original payload, the error code, the processing details and the status. This dead-letter queue preserves rejected data for investigation and supports reconciliation.

Update policies connect the Bronze source tables to the parsing and rejection functions, so that processing occurs automatically as new data arrives. The clear-table commands provided with the KQL scripts are intended for controlled testing or reset scenarios and are not part of routine production processing.

10.4 The Three Detectors

The detection layer evaluates validated telemetry against operational rules. A shared Episodes function groups consecutive readings that satisfy a condition and determines when the configured duration or reading-count requirement has been met. This reduces the likelihood of raising an alert for a single transient reading. The three detectors are as follows.

- **Cold-chain breach:** identifies cargo temperature outside the allowed range for the configured hold period. Key inputs are the temperature readings, the trip-to-cargo mapping and the cargo limits.
- **Excessive idling:** identifies an engine that is running while the vehicle speed remains below the configured idle threshold. Key inputs are the engine state, the vehicle speed and the idle duration rule.
- **Route deviation:** identifies a vehicle location beyond the permitted distance from the route for the required number of consecutive readings. Key inputs are the GPS coordinates and accuracy, the route geometry and the deviation limit.

Each detector returns a common alert structure containing an alert key, category, type, severity, status, vehicle and trip context, condition timestamps, observed and threshold values, a message, and the arrival time of the event that triggered the alert. This shared structure allows the results to be combined by the AllAlerts() function and consumed consistently by the downstream steps.

10.5 Alert Tables and the Writer

The table alert_events is the persistent destination for generated alerts. The AllAlerts() function combines the outputs of the three detectors. The writer compares these results with the records already present in alert_events and appends the records whose combination of alert key and status is not yet stored. Detection and persistence are therefore separate: the detector functions calculate alert candidates, and the writer persists the newly identified alert records.

The silver_alerts() function provides a deduplicated, latest-record view by selecting the most recent record for each alert_key according to last_updated_at. This is the logical Silver alert dataset available for downstream consumption. The supplied KQL does not show an explicit, separate Gold-layer table or load operation. The Gold-layer load of the alerts from alert_events is performed by the notebook nb_p4_fact_alerts_load (Section 11.2), and the dashboard consumption is described in Sections 10.7 to 10.9.

10.6 Evaluation and the Parity Test

The evaluation checks whether the outcomes of the pipeline align with the test scenarios recorded in the table ground_truth. For rejected inputs, the expected dead-letter error codes are compared with the actual dlq_speed counts. For alert scenarios, the expected events are matched to the detected alerts using the trip, the alert type and a time window around the injected condition.

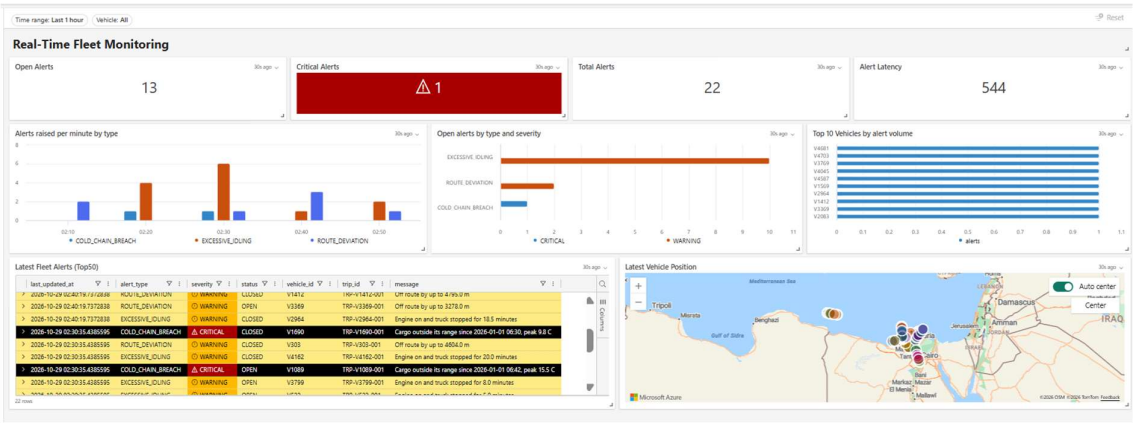
The parity check compares the raw input counts with the accepted and rejected counts of each signal. The expected relationship is raw rows = parsed rows + rejected rows. A non-zero difference indicates records that require further investigation. The alert evaluation also

calculates precision and recall to summarize the quality of the detection against the available ground truth. These metrics should be interpreted in the context of the test data and the matching rules. The results of these checks were not provided in the source material and are therefore not reported here.

10.7 Dashboard Design

The Real-Time Dashboard rtd_fleet_live has two pages and reads directly from the KQL database. It provides a time-range parameter and a vehicle parameter. The time filters use wall-clock times (detected_at and arrival_time), because the event time in the data is simulated. Each tile runs its own query on every refresh, and the dashboard contains about 15 tiles. Automatic refresh is enabled, and the refresh interval is chosen to fit the schedule of the alert writer and the capacity of the trial. The specific refresh settings were not provided in the source material.

10.8 Page 1: Live Operations



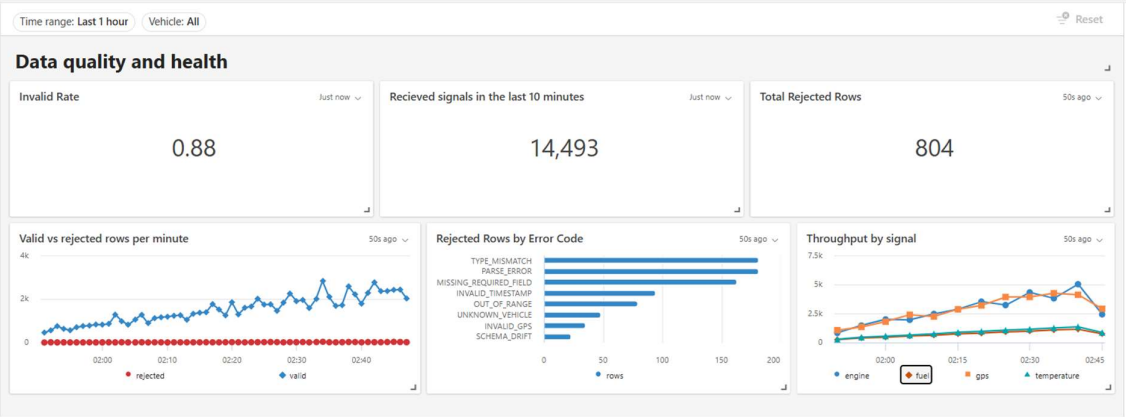
The first page supports dispatchers in acting on alerts while vehicles are still on the road. It contains the following tiles.

- **Open alerts now:** the number of conditions that are still active. This is the work queue of the dispatcher and shows how many vehicles require an intervention at the current moment. A rising number indicates that the team is falling behind.
- **Critical open:** the number of open cold-chain breaches. A refrigerated shipment outside its temperature range loses value, as recorded in the cargo table through the spoilage loss percentage, and the shipment may end in a rejected delivery. This tile justifies real-time monitoring, because each minute of delay increases the loss. Its value should remain at 0.
- **Alerts this run:** the total number of alerts raised. It indicates the scale of the operational problem and serves as the denominator for rates such as alerts per trip.
- **p95 alert latency:** the time between the arrival of the data and the writing of the alert, for 95 percent of the alerts. It measures the value of the speed layer itself, because a late alert has less business value. It covers the period from ingestion to alert and does not include the transport time before ingestion.

- **Alerts raised per minute by type:** shows when and what kind of problems occur. It supports staffing decisions and shows whether a cause, for example a hot period, affects many vehicles together.
- **Open alerts by type and severity:** splits the work queue into cold-chain alerts (critical) and idling and deviation alerts (warning), so that the team handles critical items first.
- **Latest alerts table:** provides the detail needed for action, namely the vehicle, the trip, the message and the time. It turns a count into information that a dispatcher can use to contact a driver.
- **Alerts by vehicle (top 10):** identifies repeat offenders. Many alerts on the same vehicle can indicate a faulty refrigeration unit or a driver who needs coaching, which supports maintenance and training decisions.
- **Map of last positions (optional):** shows where the fleet is and where the problems are on the road.

Impact of the page: the page moves the response from discovering a problem after delivery to acting on it while the vehicle is still on the road. It protects cargo value, reduces fuel wasted through idling, and keeps routes and delivery times under control.

10.9 Page 2: Data Quality and Health



The second page shows whether the data behind the alerts can be trusted. It contains the following tiles.

- **Invalid rate, last 10 minutes:** the share of messages rejected as invalid. Bad data leads to missing or wrong alerts, so a high rate warns that the alert numbers cannot be trusted. The tile turns red above the 2 percent rule (invalid_rate_alert_pct) and shows device or integration problems early.
- **Messages in the last minute:** the ingestion rate. A drop to zero means that a sender or the stream has stopped, which is a risk because the detectors then see no data and raise no alerts.
- **Rejected rows (total):** the cumulative size of the data-quality problem, kept for audit.

- **Valid vs rejected over time:** shows whether rejections occur in bursts, which indicates a faulty batch or device, or continuously, which indicates a systematic fault. This points to the cause.
- **Rejected rows by error code:** the reason behind the rejections. It shows which fix is worth the effort, for example correcting the device firmware or the reference data.
- **Throughput by signal:** the volume of each signal over time. A signal that disappears while the others continue points to the failure of one sensor type or one pipeline branch.
- **p95 alert latency by type:** checks whether each alert type is detected within the service level that the business requires, for example whether cold-chain alerts are detected faster than idling alerts.

Impact of the page: this page is the trust layer. It shows that the alerts on Page 1 rest on reliable data, and it shortens the time needed to find and fix a data problem.

10.10 Scheduler: pl_rt_alert_writer

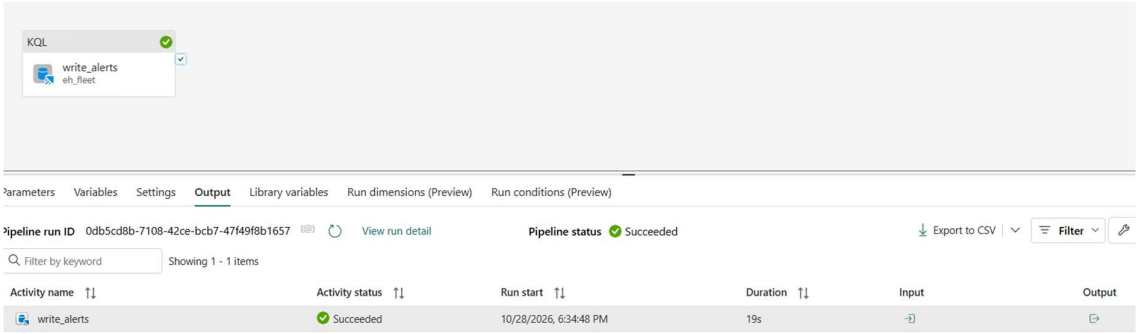


Figure 4. Succeeded pipeline run of the KQL activity write_alerts on eh_fleet

A Data Factory data pipeline schedules the KQL writer, so that the persistence of alerts does not depend on running a query manually. The pipeline is named pl_rt_alert_writer and contains a KQL activity named write_alerts. A succeeded run of the write_alerts activity on eh_fleet is shown in Figure 4 (Section 11.2). The pipeline is set up with the following procedure.

1. In the workspace, select New item, then Data pipeline, and name it pl_rt_alert_writer.
2. Add a KQL activity and name it write_alerts.
3. In the activity settings, select the KQL database eh_fleet as the connection.
4. Test the connection and the command execution with the harmless command .show tables. Save and run the pipeline. If the command is rejected by the activity, use the alternative supported by the platform for executing KQL control commands.
5. Replace the test command with the alert-writer query given below and set the activity timeout to 00:03:00.
6. Run the activity manually once and verify that alert_events contains records, for example by running alert_events | count.

7. Configure the schedule of the pipeline according to the required refresh frequency, then monitor the run history and failures.

The alert-writer query is as follows.

```
.append alert_events <|
    AllAlerts()
    | join kind=leftanti (alert_events | project alert_key, status) on alert_key, status
    | extend detected_at = now(), last_updated_at = now()
    | project alert_key, alert_category, alert_type, severity, status,
    vehicle_id, trip_id, shipment_id, driver_id,
    condition_start_time, condition_met_time, closed_at,
    observed_value, threshold_value, value_unit, detected_by,
    message, trigger_arrival_time, detected_at, last_updated_at
```

The left-anti join prevents the writer from appending an alert record when the same combination of alert_key and status already exists. The timeout limits how long a run can remain active and helps to prevent a stalled execution from delaying subsequent scheduled runs. The schedule itself should be selected according to the required freshness of the alerts and the expected event volume; the selected frequency was not provided in the source material.

11. Gold Layer

The notebook nb_p3_batch_job_2 transforms Silver data into the Gold layer. It builds the dimensions, the fact tables (including fact_trip_performance, fact_cold_chain and dim_vehicle) and the dashboard tables. It contains assertions that fail the notebook, and therefore the pipeline, when the data is wrong; these were kept deliberately as a quality gate. The Gold tables are created with CREATE TABLE IF NOT EXISTS, and Gold tables are written by overwrite, which makes retries safe.

11.1 Dimensional Model

Figure 5 shows the galaxy schema of the Gold layer, in which several fact tables share a set of conformed dimensions.

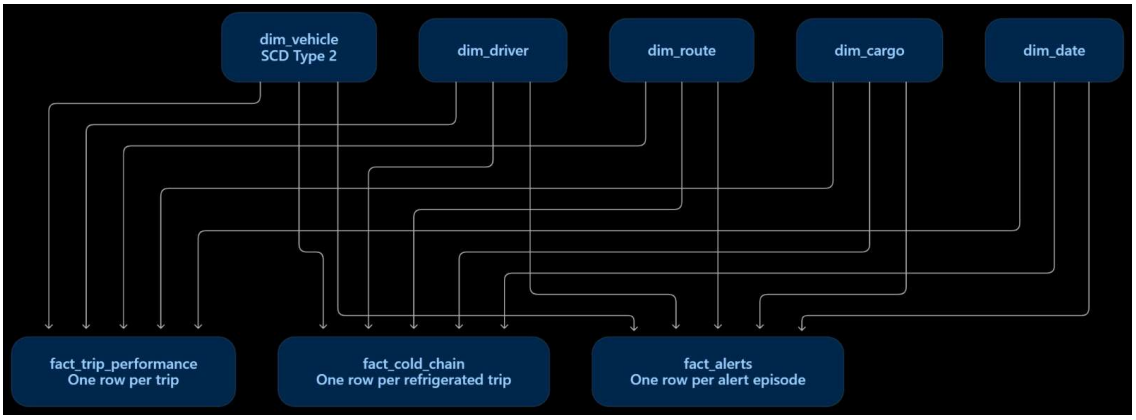


Figure 5. Galaxy schema of the Gold layer

The dimension tables are dim_vehicle (slowly changing dimension Type 2), dim_driver, dim_route, dim_cargo and dim_date. The fact tables are as follows.

- fact_trip_performance: one row per trip.
- fact_cold_chain: one row per refrigerated trip.
- fact_alerts: one row per alert episode.

The Gold guide describes the trip-level table as containing one row per trip (21 rows in the smoke data), covering measures such as fuel, idle minutes, cargo temperature compliance, late or on-time status, and a driver score. Detailed column-level definitions of the Gold tables were not available in the source material and are therefore not reproduced here.

11.2 Alerts Fact

The notebook nb_p4_fact_alerts_load reads the KQL table alert_events and takes the latest state per alert_key (using arg_max). It joins the result with ref.ref_trips (route and cargo type) and with gold.dim_vehicle when that table exists. It then merges the data into gold.fact_alerts: rows with source_system = 'KQL' are updated, and rows written by the batch jobs are never touched. The first load produced 6 alerts (COLD_CHAIN_BREACH 1, EXCESSIVE_IDLING 2 and ROUTE_DEVIATION 3). Figure 4 shows a succeeded pipeline run of a KQL activity named write_alerts that targets eh_fleet; the run started on 28 October 2026 at 6:34:48 PM and lasted 19 seconds.

12. Power BI Dashboard Development and Business Insights

12.1 Dashboard Overview

The Power BI dashboard provides a business-facing reporting layer for the Fleet Telematics Lakehouse. It transforms processed fleet telemetry and operational metrics into visual reports that help users understand fleet performance, fuel efficiency, cold-chain compliance, driver performance, and operational alerts.

The dashboard complements the real-time monitoring solution described in Section 10. While the Real-Time Dashboard (rtd_fleet_live) reads directly from the KQL database and supports live operational monitoring, the Power BI report focuses on analyzing fleet performance through aggregated metrics, historical comparisons, and detailed records.

The report is organized into five pages:

- **Overview:** fleet summary and trip distribution.

- **Fuel:** fuel efficiency by route and over time.
- **Cold Chain:** temperature compliance, coverage, breaches, and cargo value at risk.
- **Drivers:** driver performance scores and contributing operational factors.
- **Alerts:** alert severity, distribution by route, and detailed alert records.

Together, these pages provide a structured view of fleet operations, from overall performance to specific areas requiring investigation.

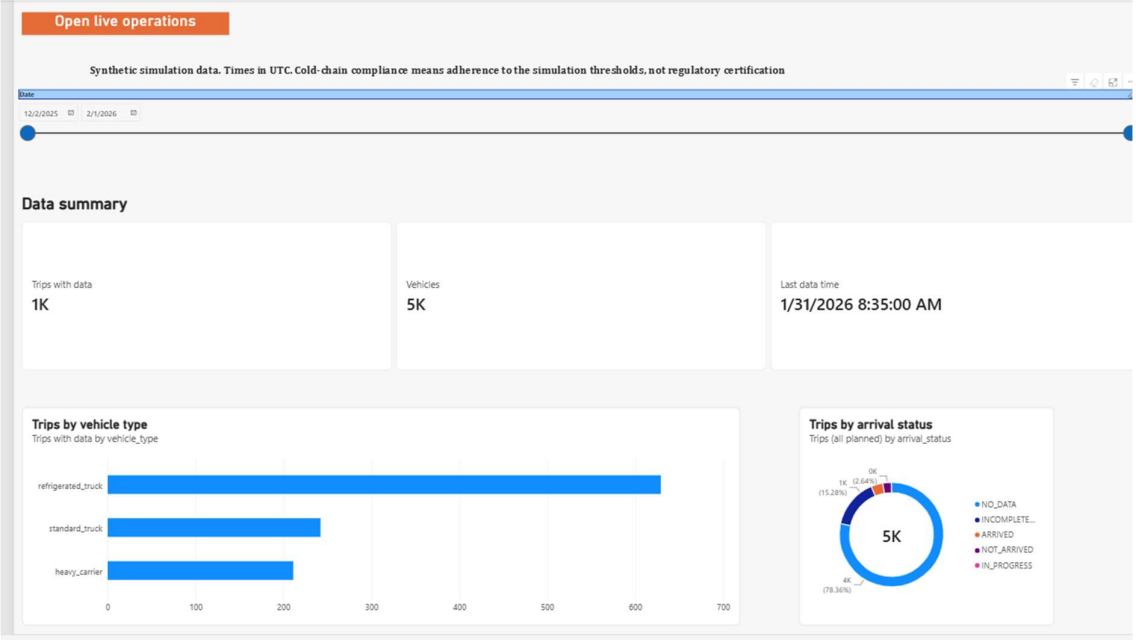


Figure 6. Fleet Telematics Power BI report — Overview page.

12.2 Business Problem and Objectives

Fleet operators need to monitor several operational dimensions simultaneously. Reviewing raw GPS, engine, fuel, and temperature events individually would make it difficult to identify patterns, compare performance, and prioritize operational improvements.

The dashboard addresses this problem by presenting processed telemetry through business-oriented metrics and visualizations.

Its main objectives are to:

1. Summarize fleet activity and trip outcomes.
2. Compare fuel efficiency across routes.
3. Monitor cold-chain compliance and estimate potential cargo losses.
4. Evaluate driver performance using a weighted score.
5. Identify the distribution and status of operational alerts.
6. Help users investigate problems by filtering the report by date and examining relevant charts and tables.

The report is intended to support operational analysis and decision-making. It does not replace the real-time alerting system or the underlying data-quality checks.

12.3 Data Flow and Power BI Integration

The Power BI report is part of the serving layer of the Fleet Telematics Lakehouse.

The documented data architecture consists of the following main stages:

1. **Telemetry generation:** A seeded Python simulator generates GPS, engine, fuel, and temperature events.
2. **Streaming ingestion:** Fabric Eventstreams send the signals to the Eventhouse.
3. **Bronze layer:** Raw messages are retained in the raw ingestion tables and copied into Lakehouse Bronze Delta tables.
4. **Silver layer:** Spark notebooks validate, type, enrich, and deduplicate the telemetry. Invalid records are recorded in the dead-letter queue.
5. **Gold layer:** The batch job creates dimensions, fact tables, and dashboard-oriented data structures.
6. **Serving and visualization:** Power BI presents the resulting business metrics through report pages.

The Gold dimensional model includes the following documented tables:

- gold.dim_vehicle
- gold.dim_driver
- gold.dim_route
- gold.dim_cargo
- gold.dim_date
- gold.fact_trip_performance
- gold.fact_cold_chain
- gold.fact_alerts

The dimensions provide descriptive context, while the fact tables contain records and measures used to analyze trips, cold-chain performance, and alerts.

The exact Power BI connection mode, individual visual-to-table mappings, and complete semantic model relationships should be documented from the actual Power BI model. They should not be assumed solely from the existence of the Gold tables.

12.4 Dashboard Design and Navigation

The report uses five pages, each addressing a different business question. A date-range slicer is visible on the report pages, allowing users to examine data within a selected period.

The Overview page provides the starting point. Users can then navigate to Fuel, Cold Chain, Drivers, or Alerts to investigate specific operational dimensions.

The report also includes an **Open live operations** button on the Overview page. This provides a navigation point toward the real-time monitoring experience.

The Overview page displays the following disclaimer:

Synthetic simulation data. Times in UTC. Cold-chain compliance means adherence to the simulation thresholds, not regulatory certification.

This distinction is important because the project uses simulated telemetry and configured business rules. Its compliance measures demonstrate the operation of the analytical pipeline rather than certify compliance with an external regulatory standard.

12.5 Overview Page

12.5.1 Purpose

The Overview page summarizes fleet activity and provides a starting point for further analysis.

It displays the following headline metrics:

- **Trips with data:** 521.
- **Vehicles:** displayed as 5K.
- **Last data time:** 1/31/2026 7:03:00 AM.

The page also contains a chart showing trips by vehicle type and a chart showing the distribution of trips by arrival status.

12.5.2 Trips by Vehicle Type

The bar chart compares trip counts across vehicle categories, including refrigerated trucks, standard trucks, and heavy rigid vehicles.

This visualization helps users understand how trip activity is distributed across the fleet's vehicle types.

It also provides context for interpreting other metrics. For example, refrigerated vehicles are particularly relevant to cold-chain monitoring, while comparisons across vehicle categories can help explain differences in operating patterns.

12.5.3 Trips by Arrival Status

The arrival-status visualization groups trips according to their recorded arrival classifications.

It helps users investigate whether trips were completed within the expected time or fell into another recorded status.

Arrival-status categories should be interpreted according to the definitions and grace-period rules implemented in the data model.

12.5.4 Business Value

The Overview page gives users a high-level view before they investigate individual areas of performance. It combines fleet-scale information with trip distribution and provides navigation to more detailed analysis.

12.6 Fuel Analysis Page

12.6.1 Purpose

The Fuel page examines fuel efficiency across routes and over time.

Fuel efficiency is important because fuel consumption contributes directly to fleet operating costs. Comparing routes can help identify differences in fuel usage and provide a basis for further investigation.

The page contains:

- A date-range slicer.
- A bar chart titled **Fuel per 100 km by route (lower is better)**.
- A table titled **Distance and fuel by route**.
- A card titled **Overall fuel per 100 km**.
- A chart titled **Fuel per 100 km over time**.

12.6.2 Overall Fuel Efficiency

The dashboard displays an overall fuel-efficiency value of **28.14 per 100 km**.

The metric is intended to express fuel consumption relative to distance travelled. Lower values indicate better fuel efficiency, assuming the underlying measure uses consistent units and correctly aggregated fuel and distance values.

This ratio-of-sums approach is preferable to taking a simple average of individual trip-level fuel-efficiency values when trips have different distances.

The precise units and measure implementation should be verified in the Power BI semantic model.

12.6.3 Fuel Efficiency by Route

The route comparison chart allows users to compare the fuel-efficiency measure across the available routes.

The accompanying table provides additional context by displaying distance, fuel consumption, trips with fuel data, and route name.

These two visuals serve complementary purposes: the chart supports rapid comparison, while the table provides more detail for investigating individual routes.

12.6.4 Fuel Efficiency Over Time

The time-series chart is intended to show how fuel efficiency changes over the selected period.

With a sufficiently populated time series, users could investigate whether fuel consumption changes over time or whether particular operating periods require further analysis.

However, the chart should only be used to draw trend conclusions when enough valid observations are available. A single or sparse set of points does not establish a meaningful trend.

12.6.5 Business Questions

The Fuel page supports questions such as:

- Which routes have higher fuel consumption per 100 km?
- How much distance and fuel consumption are associated with each route?
- How many trips contribute fuel data to the analysis?
- Does fuel efficiency change over time?

The dashboard identifies where further investigation may be useful. It does not, by itself, establish why a route consumes more fuel.

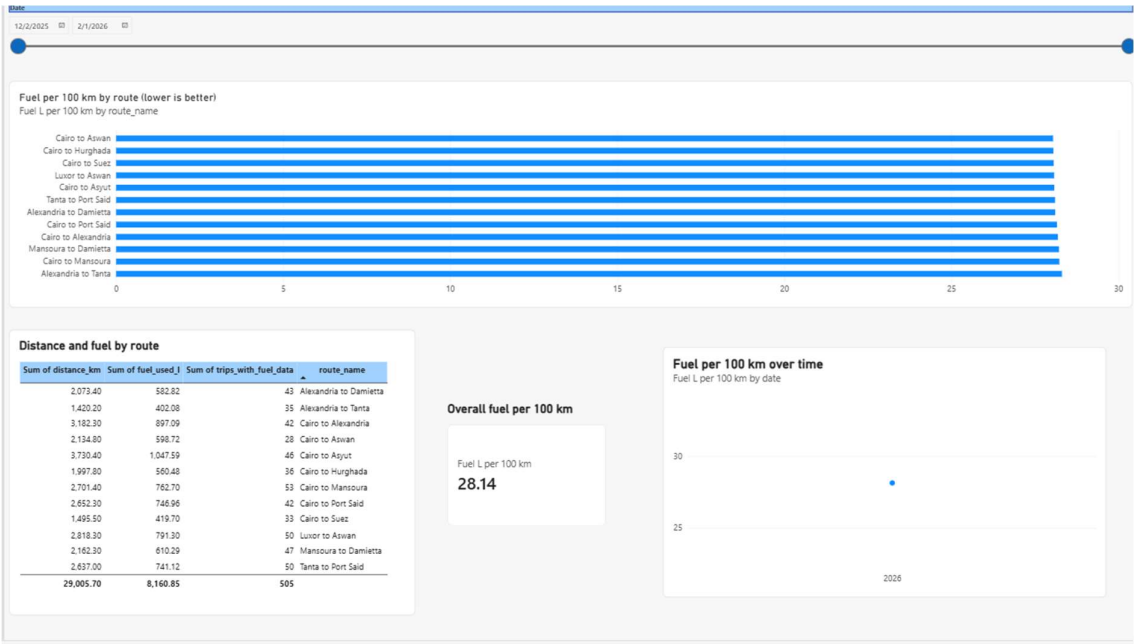


Figure 7. Fuel-efficiency analysis by route and over time.

12.7 Cold-Chain Analysis Page

12.7.1 Purpose

The Cold Chain page focuses on the temperature conditions of shipments requiring controlled-temperature transportation.

Temperature excursions can threaten cargo quality and value. Monitoring compliance, measurement coverage, breach events, and estimated cargo risk helps identify potential operational problems.

The page displays four headline metrics:

Metric	Displayed value
Cold compliance %	91.00%
Cold coverage %	6.62%
Breach trips	6
Cargo at risk (EGP)	248.50K

These measures describe different aspects of cold-chain performance and should not be interpreted as interchangeable.

12.7.2 Cold Compliance

Cold compliance represents the proportion of judged temperature time that falls within the configured acceptable range.

The compliance metric evaluates readings that can be judged against the configured temperature thresholds.

12.7.3 Cold Coverage

Cold coverage indicates how much of the relevant observation period has usable temperature measurements.

Coverage is essential when interpreting compliance. A high compliance percentage does not necessarily mean that the full shipment history was monitored.

Sensor faults, missing measurements, or other periods without usable readings should not automatically be treated as compliant. They represent a limitation in the available evidence.

The displayed coverage value of 6.62% therefore deserves particular attention. Users should examine measurement completeness before drawing strong conclusions from the compliance percentage.

12.7.4 Breach Trips

The Breach trips KPI displays **6**.

According to the documented business logic, a breach trip is a trip with an out-of-range temperature episode lasting at least the configured alert hold time.

The duration threshold helps distinguish persistent temperature excursions from brief fluctuations that do not meet the alerting condition.

The exact relationship between this KPI and the real-time detector should be verified against the relevant measures and source tables.

12.7.5 Cargo at Risk

The dashboard displays **EGP 248.50K** as cargo at risk.

The documented business logic estimates risk using cargo value and the applicable spoilage fraction, and counts the risk when the trip is considered compromised under the configured excursion-duration rules.

This is an estimated financial exposure, not proof that the cargo was actually damaged or that a loss occurred.

12.7.6 Compliance by Cargo Type

The chart titled **Compliance by cargo type (%)** compares compliance across cargo categories.

It helps users investigate whether particular cargo types experience more temperature-control problems than others.

The displayed categories include Cold Medicine, Chilled Food, and Frozen Food. Their results should be interpreted alongside the coverage metric, since incomplete measurements can limit the reliability of comparisons.

12.7.7 Compliance by Route

The report contains a chart titled **Compliance by route (%)**. However, the displayed value field is Sum of cargo_at_risk by route_name, with a value axis reaching approximately 200K.

This indicates a mismatch between the chart title and the displayed measure.

Until the Power BI configuration is corrected, this chart should **not** be interpreted as route-level cold-chain compliance.

Two corrections are possible:

1. Replace the displayed measure with a properly defined route-level compliance percentage and retain the current title.
2. Retain the cargo-at-risk measure and rename the chart to **Cargo value at risk by route (EGP)**.

The final choice should reflect the intended business question and the measures available in the semantic model.

12.7.8 Business Value

The Cold Chain page combines compliance, coverage, breach counts, and estimated financial risk. These measures allow users to investigate temperature-control performance while distinguishing confirmed measurement results from periods where data is insufficient.

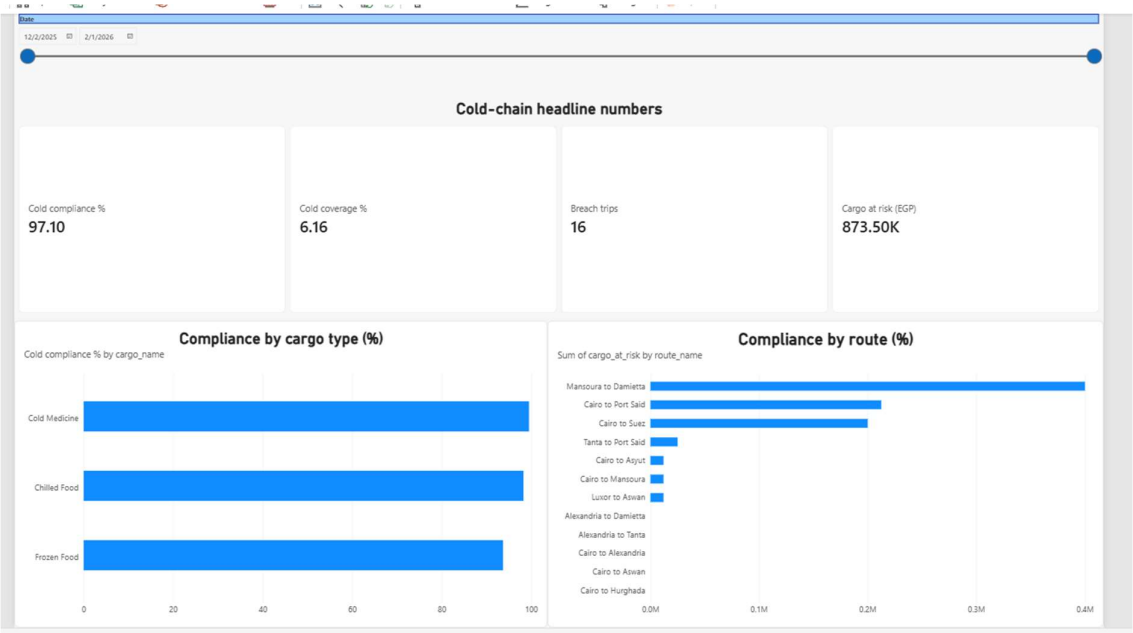


Figure 8. Cold-chain compliance and cargo-risk analysis.

12.8 Driver Performance Page

12.8.1 Purpose

The Drivers page evaluates driver performance through a weighted score and supporting operational measures.

The page contains:

- A KPI card titled **Driver score (weighted)**.
- A detailed table titled **Driver scorecard**.
- A chart titled **Top 10 drivers**.
- A chart titled **Bottom 10 drivers**.
- A date-range slicer.

The displayed weighted driver score is **80.16**.

12.8.2 Weighted Driver Score

The documented scoring design produces a score from 0 to 100. It considers several driving-performance factors, including speeding, excessive idling, route deviations, and late trips.

The documented penalty weights are:

Factor	Weight
Speeding	35
Excessive idling	30
Route deviation	20
Late trips	15
Total	**100**

Each penalty is capped at its assigned weight.

The intended score bands are:

- **Good:** 85 or above.
- **Watch:** 70 or above but below 85.
- **At risk:** below 70.

The detailed scorecard displays the driver name, weighted score, score band, excessive-idling minutes, speeding minutes, route-deviation episodes, and late-trip information.

These supporting fields help users understand which operational factors contribute to a driver's result.

The exact weighting and aggregation implemented in Power BI should be checked against the model or calculation logic before these definitions are treated as a verified description of the live report.

12.8.3 Top 10 and Bottom 10 Drivers

The Top 10 and Bottom 10 charts rank drivers according to their displayed scores.

The Top 10 chart highlights drivers with stronger scores under the configured scoring model. The Bottom 10 chart identifies drivers whose scores may warrant further review.

The charts are intended to support investigation, coaching, and performance monitoring. A low score should not automatically be interpreted as evidence of misconduct, because the score depends on the quality, completeness, and attribution of the underlying data.

12.8.4 Business Value

The Drivers page consolidates several performance indicators into a comparable score and provides supporting detail for interpretation.

It helps answer questions such as:

- Which drivers have the highest and lowest scores?
- Which operational factors are associated with lower scores?
- Which drivers may benefit from additional investigation or coaching?

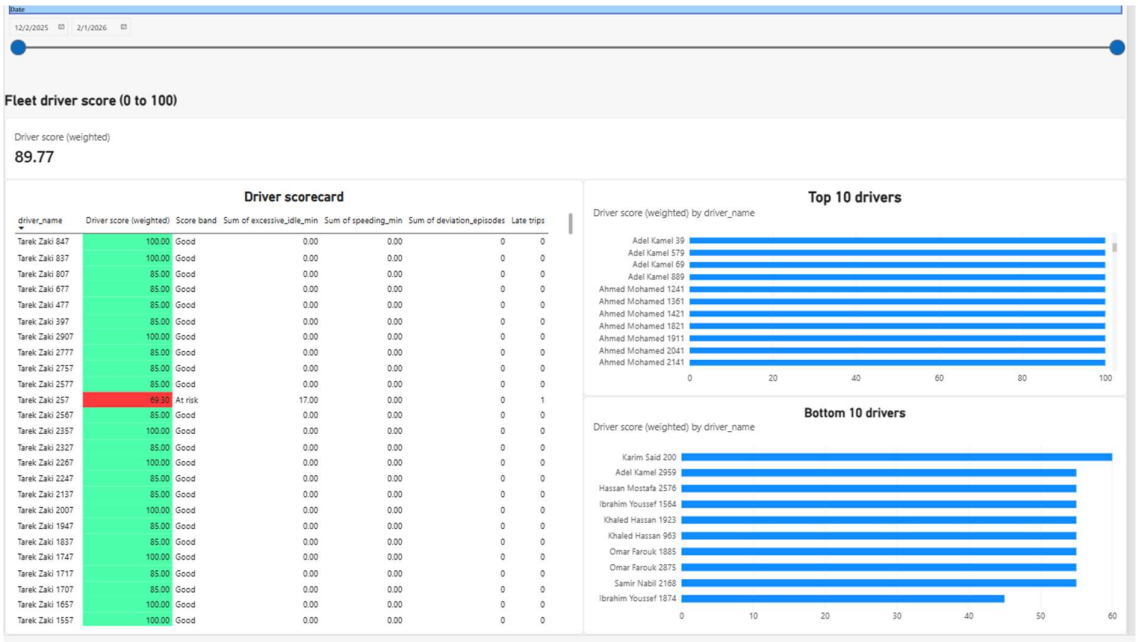


Figure 9. Driver performance scorecard and driver rankings.

12.9 Alerts Analysis Page

12.9.1 Purpose

The Alerts page provides a historical and analytical view of alert records.

It complements the real-time dashboard, which is designed for monitoring active operational conditions. The Power BI page supports reviewing alert severity, route distribution, detector information, and individual alert records.

The page contains:

- **Alerts by severity**.
- **Alerts by route**.
- **Alert log**.
- **Alerts by detector**.
- A date-range slicer.

12.9.2 Alerts by Severity

The severity chart displays **28 alerts**, distributed as follows:

Severity	Count	Percentage
WARNING	25	89.29%
CRITICAL	3	10.71%
Total	**28**	**100%**

This distribution indicates that warning-level alerts make up most of the displayed records, while a smaller proportion are critical.

Severity helps users prioritize investigation. However, the number of alerts alone does not establish the financial impact or seriousness of every event; users should also consider alert type, status, affected vehicle, and cargo context.

12.9.3 Alerts by Route

The route chart displays the following alert counts:

Route	Alert count
Tanta to Port Said	8
Cairo to Mansoura	7
Mansoura to Damietta	4
Cairo to Alexandria	3
Alexandria to Damietta	2
Alexandria to Tanta	2
Cairo to Port Said	2
Total displayed	**28**

The chart helps identify routes with higher alert counts within the selected reporting period.

A higher count may justify further investigation, but it does not prove that a route is inherently riskier. A fair comparison would also consider the number of trips, distance travelled, operating conditions, and the distribution of vehicles across routes.

12.9.4 Alert Log

The alert log provides event-level detail. The visible columns include:

- alert_type
- severity
- status
- vehicle_id
- trip_id
- opened_at
- closed_at
- message

Examples of alert types include ROUTE_DEVIATION, EXCESSIVE_IDLING, and COLD_CHAIN_BREACH.

The message field provides context about the detected condition, such as a vehicle leaving its route, remaining stopped with its engine running, or cargo temperature moving outside the configured range.

The status field distinguishes records marked OPEN from those marked CLOSED.

This table is useful when a summary chart identifies an area requiring investigation and the user needs to inspect the underlying alert records.

12.9.5 Alerts by Detector

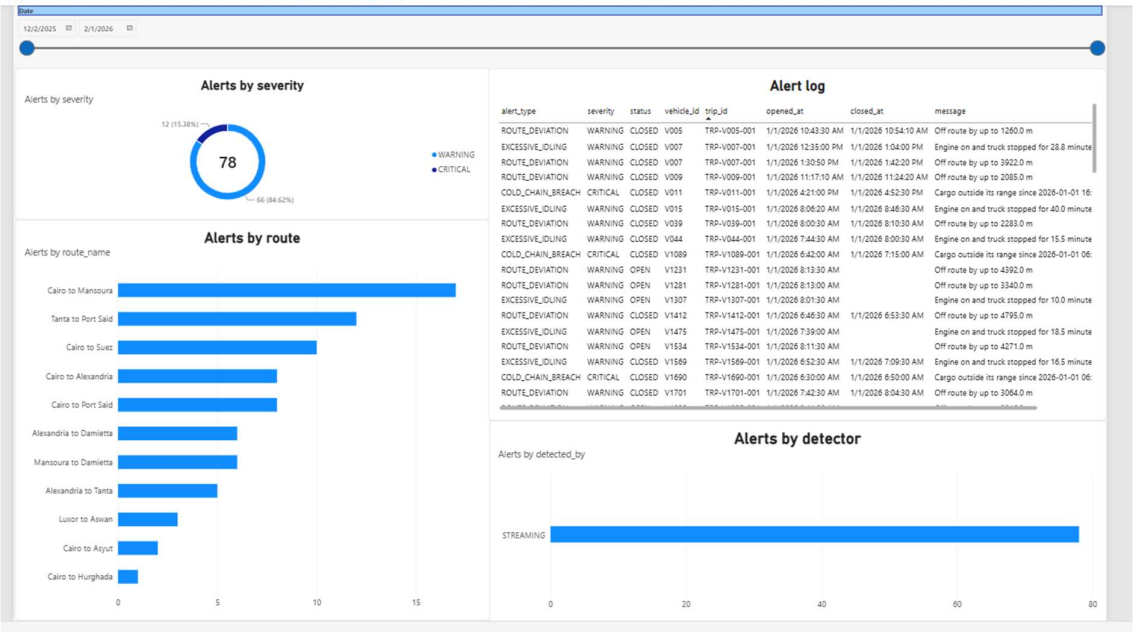
The chart titled **Alerts by detector** shows STREAMING as the visible detector category, accounting for the displayed alert records.

This describes the detector attribution in the current report. It should not be interpreted as evidence that every processing component in the overall project operates exclusively in real time.

The project also contains separate batch processing and alert-loading stages.

12.9.6 Business Value

The Alerts page connects aggregate alert patterns with detailed event records. It helps users determine which routes have the most alerts, which severity levels dominate, and which specific events m



ay require further review.

Figure 10. Historical alert distribution and event-level details.

12.10 How the Dashboard Pages Work Together

The five report pages provide complementary views of fleet operations.

Page	Main question	Primary business value
Overview	What does overall fleet activity look like?	Establishes operational context.
Fuel	How does fuel efficiency vary by route and time?	Helps identify opportunities to investigate fuel consumption.
Cold Chain	How well are temperature conditions maintained, and what cargo value may be at risk?	Highlights temperature-control and measurement-coverage concerns.

Drivers	How do drivers compare under the configured performance score?	Supports investigation and coaching.
Alerts	Where and how frequently are operational alerts occurring?	Supports prioritization and investigation of detected conditions.

For example, an elevated alert count on a particular route can prompt further analysis of that route's fuel performance or cold-chain results. The driver scorecard may then provide additional operational context if the relevant records can be linked correctly.

Such cross-page analysis should be based on compatible filters, relationships, and measures. A visual correlation does not, by itself, prove that one factor caused another.

12.11 Dashboard Findings and Interpretation

The displayed report provides several observations that can be reported directly.

12.11.1 Fleet Activity

The Overview page displays 521 trips with data and a vehicle KPI shown as 5K. These values summarize fleet activity as represented by the report's current model and filters.

The precise meaning of the vehicle KPI should be confirmed in the measure definition before describing it as a distinct vehicle count or another specific aggregation.

12.11.2 Fuel Efficiency

The overall fuel-efficiency KPI is 28.14 per 100 km. The route chart and detailed table provide the means to investigate differences in fuel usage between routes.

A conclusion about the most or least efficient route should be based on the route-level values and the calculation used by the report.

12.11.3 Cold-Chain Performance

The displayed compliance is 91.00%, while coverage is 6.62%. The combination is important: compliance describes the judged temperature observations, while coverage describes the availability of usable measurements.

The dashboard also reports six breach trips and EGP 248.50K in cargo at risk. These values indicate the measures presented by the report; they do not independently establish actual cargo damage or realized financial loss.

The route chart's measure mismatch must be resolved before route-level compliance conclusions can be drawn from that visual.

12.11.4 Driver Performance

The weighted driver score is 80.16. The scorecard and ranking charts provide the detailed comparisons required to identify higher- and lower-scoring drivers.

Specific conclusions about individual drivers should be based on the displayed scorecard records and verified scoring rules.

12.11.5 Alert Distribution

The report displays 28 alerts: 25 warnings and three critical alerts. The route chart shows that Tanta to Port Said has the highest displayed count, with eight alerts, followed by Cairo to Mansoura with seven.

These counts identify where further investigation may be useful. Route-level rates would be needed to make stronger comparisons that account for different trip volumes.

12.12 Limitations and Recommended Improvements

The dashboard provides useful business views, but several areas should be reviewed before treating every visualization as a definitive operational measure.

12.12.1 Validate Measures Against the Data Model

Confirm that every KPI and chart uses the intended Gold table, column, measure, aggregation, and filter context.

In particular, verify the definition of the vehicle KPI and the exact implementation of the fuel-efficiency and driver-score measures.

12.12.2 Correct the Cold-Chain Route Chart

The chart titled **Compliance by route (%)** currently displays Sum of cargo_at_risk by route_name.

The measure or title must be corrected so that the chart communicates one consistent business concept.

12.12.3 Improve Interpretation of Cold-Chain Coverage

Because coverage is displayed separately from compliance, users should be encouraged to consider both measures when interpreting temperature performance.

Future improvements could include showing the amount of measured and unmeasured time or providing a detailed view of trips with insufficient temperature observations, provided the required data and measures are available.

12.12.4 Strengthen Fuel Trend Analysis

A time-series chart is most useful when it contains enough observations to reveal a meaningful pattern. Verify that the date field, measure, and aggregation produce a useful trend at the intended reporting granularity.

12.12.5 Add Alert Summary Measures

The current screenshot shows severity, route, detector, and event-level alert information. Additional cards for total alerts, open alerts, and critical alerts could improve the summary view if these measures are available and correctly defined.

These would be enhancements, not features confirmed in the current screenshots.

12.12.6 Clarify Units and Definitions

Ensure that fuel, distance, cargo risk, time, and score measures have clear units and definitions. The report's synthetic-data disclaimer should remain visible so users understand the scope of the simulation.

12.12.7 Verify Filter and Relationship Behaviour

Test that date filters and any shared dimension filters affect the intended charts and tables consistently. Confirm that the Power BI model relationships support the comparisons users are expected to make across pages.

12.13 Conclusion

The Power BI dashboard provides an analytical interface for the Fleet Telematics Lakehouse. Its five pages summarize fleet activity, fuel efficiency, cold-chain performance, driver scores, and operational alerts.

The report transforms processed telemetry into business-oriented measures that are easier to interpret than raw event records. It complements the project's real-time monitoring solution by supporting historical review, comparisons, and investigation.

The dashboard should be considered alongside the underlying data-quality controls, business-rule definitions, and Gold-layer model. Validating measures and correcting the identified cold-chain route-chart mismatch will help ensure that its visualizations communicate the intended business meaning accurately.

13. Batch Orchestration

13.1 Decision and Execution Order

The batch steps are orchestrated by the Data Factory data pipeline pl_fleet_batch in the workspace SIC8-Fleet-Telematics (implemented on 28 October 2026). A Data Pipeline was chosen over an Apache Airflow Job. Both can run notebooks, but the pipeline is simpler, is built into Fabric, and supports retry, timeout, schedule and If Condition without writing DAG code.

The execution order is: bronze_load, silver_setup, Wait1 (120 seconds), batch_job_1, If Condition1 (Gate 1), Wait1_copy1 (120 seconds), alerts_load and batch_job_2. Figure 11 shows a successful run of the pipeline. All activities are connected with the condition On success. No notebook takes parameters.

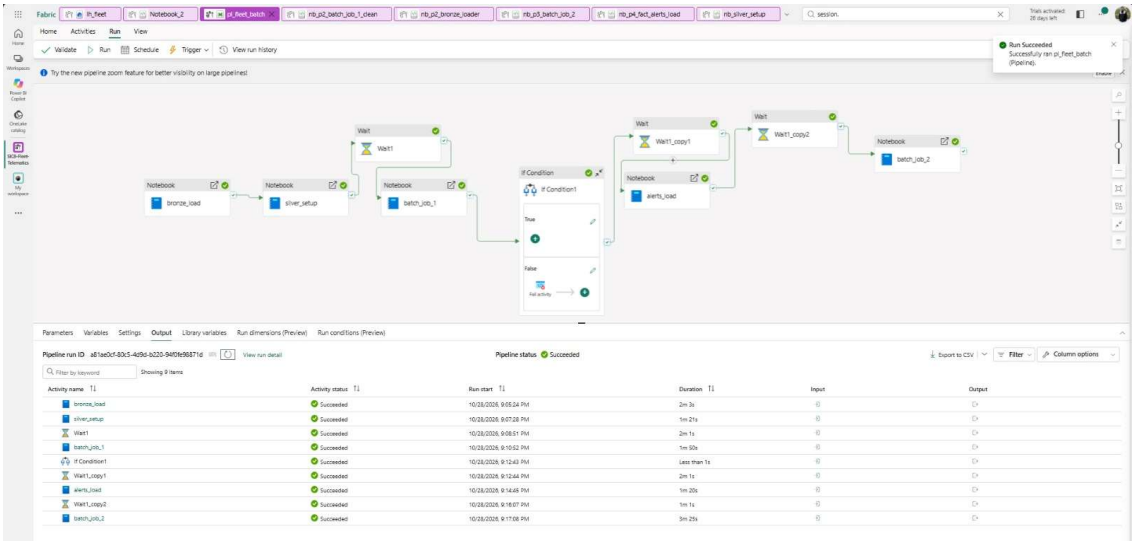


Figure 11. Successful run of the pipeline pl_fleet_batch

The activities, their notebooks, timeouts and retries are as follows. The retry interval is fixed at 60 seconds, and all activities have one retry.

- bronze_load: nb_p2_bronze_loader, timeout 0:30:00 (1:00:00 recommended).
- silver_setup: nb_silver_setup, timeout 0:15:00.
- batch_job_1: nb_p2_batch_job_1_clean, timeout 1:00:00 (1:30:00 recommended).
- alerts_load: nb_p4_fact_alerts_load, timeout 0:30:00.
- batch_job_2: nb_p3_batch_job_2, timeout 1:00:00.

All retries are safe (idempotent). Bronze has a retry guard that deletes the window of the run before rewriting it, batch_job_1 advances the watermark as its last step, alerts_load uses MERGE, and Gold uses overwrite. Raising Retry to 2 or using an increasing delay is suggested if the capacity problem continues.

13.2 Wait Activities

Two Wait activities of 120 seconds each were added as a workaround for Spark capacity: Wait1 between silver_setup and batch_job_1, and Wait1_copy1 between If Condition1 and alerts_load. Both give Spark capacity time to free up before the next session starts. Before they were added, batch_job_1 and alerts_load failed on their first attempt (after about 20 seconds) and succeeded only on retry. With the Wait activities, the run completed with no failed attempts. A Wait activity does not use Spark, so it does not compete for capacity. The cost is about 4 extra minutes per run, which is small compared with the retries it replaces. The names Wait1 and Wait1_copy1 could be changed to wait_before_batch_job_1 and wait_before_alerts_load for clarity.

13.3 Gate 1 in the Pipeline

After batch_job_1, an If Condition reads the exit value of the notebook with the expression @equals(json(activity('batch_job_1').output.result.exitValue).status, 'PASS'). The True branch is empty and the pipeline continues to alerts_load. The False branch contains a Fail activity with the message @concat('Gate 1 failed: ', activity('batch_job_1').output.result.exitValue) and an error code such as GATE1_FAILED. This condition is required because the notebook itself does not fail when Gate 1 returns FAIL (Silver is still written), so the If Condition is what stops Gold.

13.4 Schedule and Permissions

- Schedule type: fixed, repeating daily at 12:00 PM and 12:00 AM (twice a day).
- Start date 28 October 2026 and end date 28 August 2027, in the time zone UTC+02:00 (Cairo). No parameters are used.
- The 12-hour gap means that runs will not overlap. To the knowledge of the team, Fabric has no setting that prevents overlapping runs.
- An email address should be added under Failure notifications on the Schedule tab of the pipeline.
- The schedule runs under the identity of the person who created or last edited it. That identity needs the Contributor role on the workspace and Read access to the KQL database eh_fleet, because the Bronze and Alerts notebooks use its token. Different owners of the notebooks do not matter as long as this identity has Contributor access.

14. Testing, Validation and Results

14.1 Local Verification Record (26 October 2026)

- Reference data: 20 vehicles, 13 drivers, 3 routes, 59 checkpoints, 21 trips, 32 rules and 1 change file.
- Kafka smoke run (P1-10 code): 71,455 messages sent, 71,455 delivered, 0 errors (gps 29,737, engine 29,774, fuel 4,985, temperature 6,959); 71,133 built.
- Clean run (P1-11 code): 70,024 messages (gps 29,128, engine 29,128, fuel 4,862, temperature 6,906); only SHORT_STOP planned; PROBLEMS: none.
- Smoke run (P1-11 code, local): 74,194 messages (gps 30,895, engine 30,926, fuel 5,176, temperature 7,197); 74,036 parsed and 158 unparseable; 73,866 built; PROBLEMS: none.
- Anomalies: breach V011, blip V013, idling V015 and V007, deviation V005, V009 and V007, wander V012 and V006; all found.
- Late arrivals: LATE_ARRIVAL for V010 (45.3 minutes beyond grace), V001 (22.3) and V015 (9.2, side effect); LATE_WITHIN_GRACE for V002, V019, V017 and V020 (9.7 to 22.3 minutes after the planned end, grace 30); all found.
- Data faults: all 17 types were found as logged (for example DROPPED 190, DUPLICATE 373, MALFORMED 158 and MISSING_FIELD 145).
- Determinism: the same seed run twice gives an identical stream_messages.jsonl (True).
- audit_phase1.py: 28 PASS and 1 FAIL (README.md missing; the check clears when this document is saved as README.md).

14.2 Silver Results for the Smoke Run

The smoke run contained 74,194 Bronze messages. For every signal the balance Bronze = Silver + DLQ + duplicates removed holds. The counts per signal are as follows.

- gps: Bronze 30,895, Silver 30,472, DLQ 275, duplicates removed 148, invalid share 0.890 percent.
- engine: Bronze 30,926, Silver 30,512, DLQ 271, duplicates removed 143, invalid share 0.876 percent.
- fuel: Bronze 5,176, Silver 5,105, DLQ 30, duplicates removed 41, invalid share 0.580 percent.
- temperature: Bronze 7,197, Silver 7,105, DLQ 51, duplicates removed 41, invalid share 0.709 percent.
- Total: Bronze 74,194, Silver 73,194, DLQ 627, duplicates removed 373, invalid share 0.845 percent.

The DLQ rows by error code (gps / engine / fuel / temperature / total) are as follows.

- PARSE_ERROR: 73 / 51 / 14 / 20 / 158.
- MISSING_REQUIRED_FIELD: 53 / 74 / 6 / 12 / 145.

- TYPE_MISMATCH: 51 / 56 / 8 / 11 / 126.
- OUT_OF_RANGE: 35 / 36 / 0 / 0 / 71.
- INVALID_GPS: 23 / 0 / 0 / 0 / 23.
- INVALID_TIMESTAMP: 21 / 27 / 1 / 1 / 50.
- UNKNOWN_VEHICLE: 13 / 22 / 1 / 5 / 41.
- SCHEMA_DRIFT: 6 / 5 / 0 / 2 / 13.
- Total: 275 / 271 / 30 / 51 / 627.

The flag values present in the Silver tables after the run are SCHEMA_DRIFT (13 rows) in gps, and UNIT_CONVERTED (304 rows), SENSOR_NULL and SENSOR_FAULT (54 rows) and SCHEMA_DRIFT (4 rows) in temperature. Engine and fuel have no flags.

The statistics of lateness_sec per Silver table, in seconds (minimum / median / 99th percentile / maximum), are: silver_gps (30,472 rows) 1.138 / 1.552 / 1.783 / 2.562; silver_engine (30,512 rows) 1.138 / 1.552 / 1.784 / 2.000; silver_fuel (5,105 rows) 1.656 / 1.807 / 1.939 / 2.047; silver_temperature (7,105 rows) 1.150 / 1.761 / 1.801 / 1.869. All rows have a non-NULL value. All values are positive and within a few seconds, which is the expected size of the ingestion delay of a clean smoke run.

14.3 Pipeline Runs

- First successful run: total duration about 16 minutes (7:22 PM to 7:38 PM). Durations: bronze_load 2m 55s, silver_setup 1m 21s, batch_job_1 1m 49s (after a failed attempt of 19s), If Condition1 under 1s, alerts_load 1m 20s (after a failed attempt of 20s), batch_job_2 3m 39s. The failures look like a Spark capacity shortage at session start rather than a code problem, which is to be confirmed in the Output message of the failed attempt.
- Second run (with the Wait activities): succeeded with no failed attempts and no retries, in about 14 minutes (9:22 PM to about 9:36 PM). Durations: bronze_load 1m 39s, silver_setup 1m 6s, Wait1 2m 1s, batch_job_1 1m 38s, If Condition1 under 1s, Wait1_copy1 2m 1s, alerts_load 1m 16s, batch_job_2 3m 55s. This run also confirms that rerunning the pipeline is safe.
- Run shown in Figure 11: all nine activities succeeded on 28 October 2026 (bronze_load 2m 3s, silver_setup 1m 21s, Wait1 2m 1s, batch_job_1 1m 50s, If Condition1 under 1s, Wait1_copy1 2m 1s, alerts_load 1m 20s, Wait1_copy2 1m 1s, batch_job_2 3m 25s). This run includes a third Wait activity, Wait1_copy2, placed before batch_job_2.

14.4 Fabric Checks Pending

The following checks are run in the KQL database eh_fleet, each query alone, after the five raw tables have been cleared with .clear table <name> data. The results had not been recorded in the source documentation.

1. Row counts per raw table after a Kafka run, using a union with source tagging over the five raw tables and a summarize count by table. Expected for the P1-10 Kafka run:

gps 29,737, engine 29,774, fuel 4,985, temperature 6,959, total 71,455. Expected for a P1-11 Kafka run: gps 30,895, engine 30,926, fuel 5,176, temperature 7,197, total 74,194.

2. Broken messages are kept: after running test_malformed.py and waiting a few minutes, a query on bronze_gps_raw for raw_value containing BAD-TEST must return 2 rows (the valid control and the truncated text).
3. Unparseable rows after a smoke run: the total of rows whose raw_value does not convert to a dictionary must equal the number of MALFORMED rows in injected_events_log.csv (158).
4. Exact duplicates: the number of rows minus the number of distinct event_id values must equal the number of DUPLICATE rows of the log (373 in the P1-11 run).

The verification checklist for P1-12 requires the following results: with offline windows disabled, the smoke run still gives 74,194 messages; with offline windows enabled, verify_stream.py prints PROBLEMS: none, determinism is True for the stream and the bulk files, and OFFLINE_WINDOW and BUFFER_OVERFLOW counts match; the clean run gives 70,024 messages with no offline windows; and the bulk files are uploaded to Files/landing/offline/ and the device_status table receives the reports.

15. Operating Procedures

15.1 Setup and Local Run

The required packages are installed with pip install pyyaml numpy python-dotenv confluent-kafka, and the .env file is prepared as described in Section 4.3. The options --sink jsonl and --sink none need neither Kafka nor a .env file. A local smoke run follows these steps.

1. python generate_reference_data.py
2. python generate_stream.py --mode clean --sink jsonl --fast --out evidence_clean
3. python verify_stream.py evidence_clean reference
4. python generate_stream.py --sink jsonl --fast
5. python verify_stream.py evidence reference
6. python audit_phase1.py --root .

Determinism is checked by running the generator twice with --out run_a and --out run_b and comparing the md5 of the two stream_messages.jsonl files (and, with offline windows on, of the bulk files). The folders evidence_clean, run_a and run_b are deleted afterwards, because the audit reads the first evidence files it finds. The options of generate_stream.py are --mode, --accel, --sink kafka|jsonl|none, --fast, --reference DIR and --out DIR.

15.2 Run into Fabric

1. Upload reference/* to Files/reference (deleting the old files first) and reference_faults/ to its own folder.
2. Clear the five raw tables (Section 14.4).

3. Run python generate_stream.py, which is paced by A and prints progress every 10 seconds. The lag should stay near 0; otherwise A is lowered with --accel.
4. Reconcile the counts (Section 15.4). The generator must not be run twice against the same tables, because every run adds its messages again.
5. Upload run_manifest.json, injected_events_log.csv, trip_outcomes.csv and run_summary.json (not stream_messages.jsonl) to Files/evidence, and evidence/landing/offline/* to Files/landing/offline/ when offline windows are on.

15.3 Final Evidence Run (Five-Thousand-Vehicle Reference Data)

The full reference data (5,000 vehicles, evidence mode) must not be generated until the very end of the project, immediately before the final evidence run. Until then, the project works only with the smoke dataset (20 vehicles) and config.yaml stays on the mode "smoke". The generator generate_reference_data.py has no --mode option and reads the mode only from config.yaml; only one line of config.yaml changes between the two modes, and the file must not be replaced.

The final step starts only when all of the following are true: the pipeline works end to end on the smoke data, the clean mode run passes (no alerts and no DLQ rows), the acceleration factor A has been decided by the team, and at least 2 days remain before the trial ends (about 23 November 2026), because the run is long. The steps are as follows.

1. Keep the smoke results by renaming the local reference/ to reference_smoke/ and evidence/ to evidence_smoke/.
2. Edit one line in config.yaml to mode: "evidence" (adjusting acceleration_factor in the evidence profile if needed), and keep reference_file_faults set to false.
3. Generate the full reference data with python generate_reference_data.py.
4. Upload to Fabric: delete the old smoke files in Files/reference and upload the new folder.
5. Reset Bronze: stop the generator, clear every raw table in the KQL database (.clear table bronze_gps_raw data, and the same for bronze_engine_raw, bronze_fuel_raw, bronze_temperature_raw and bronze_device_status_raw), and confirm each with a count of 0.
6. Run python generate_stream.py, which takes the mode from config.yaml. The machine must stay on for the whole run: about 16 hours at A = 1, or about 8 hours at A = 2.
7. Verify by comparing the rows of the raw tables with messages_sent in run_summary.json, and upload the evidence files (Section 15.2).

Two rules must never be broken. First, the mode in config.yaml must match the reference data currently in Fabric at the time of any run. Second, if simulation.days is changed, the reference data must be regenerated and re-uploaded before the next run.

15.4 Reconciliation

- Messages built = messages_sent minus device_status reports plus bulk-file rows plus overflow-lost rows plus DROPPED_MESSAGE minus DUPLICATE minus NEAR_DUPLICATE.
- Rows in the raw tables (stream) = messages_sent of run_summary.json, and per signal = per_signal.
- Parsed rows plus unparseable rows = rows of the raw tables, and the unparseable count = the MALFORMED rows of the log.
- Rows of the raw tables minus distinct event_id = the DUPLICATE rows of the log.
- Expected DLQ rows from the stream = MALFORMED + MISSING_FIELD + WRONG_TYPE + INVALID_GPS + IMPOSSIBLE_VALUE + BAD_TIMESTAMP + UNKNOWN_VEHICLE + SCHEMA_DRIFT with drift_kind = unknown_version.
- Sequence gaps are explained by DROPPED_MESSAGE, MALFORMED (delivered, number unreadable) and BUFFER_OVERFLOW.
- With offline windows enabled: signal messages built = stream signal rows + bulk-file rows + rows lost by buffer overflow + P1-10 dropped messages minus DUPLICATE and NEAR_DUPLICATE copies. run_summary.json provides device_status_reports, bulk_files, bulk_file_messages and messages_lost_buffer_overflow, and messages_sent includes the device_status reports and the bursts.
- The column messages of trip_outcomes.csv counts built messages, not sent ones, and the injection_id numbers shift when features are switched on or off.

16. Challenges and Solutions

The following problems were found during development and orchestration, with the corresponding fixes.

- **Lakehouse destination loses data:** the Eventstream Lakehouse destination lost fields and silently dropped malformed messages. Solution: Bronze was moved to raw text tables in Eventhouse with TXT direct ingestion (Section 5.2).
- **Spark capacity:** a permanently running Spark session consumed the trial capacity (HTTP 430), and in the pipeline batch_job_1 and alerts_load failed on their first attempt. Solution: the Spark raw-sink approach was abandoned, and two 120-second Wait activities were added to the pipeline, after which no attempt failed.
- **Development notebook risks:** the original nb_p2_batch_job_1 was a development notebook with dangerous cells. Cell 51 wipes all Silver tables, the DLQ and the log and resets the watermarks; Cell 9 drops silver_temperature and recreates it with dq_flags as ARRAY<STRING>; Cell 48 raises a NameError; and Cell 39 runs gps on its own. Solution: a clean copy, nb_p2_batch_job_1_clean, was created with only 14 code cells, identical character for character to the original, with the exploration, DROP and DELETE cells and tests removed.

- **Repeated table creation:** CREATE TABLE for gold.fact_trip_performance and gold.fact_cold_chain had no IF NOT EXISTS, so a second run would fail. Solution: IF NOT EXISTS was added.
- **Session stop in the Bronze loader:** nb_p2_bronze_loader called session.stop() in Cell 3 and again in Cell 4 on a dead session. Solution: Cell 4 was deleted. Replacing session.stop() at the end of all notebooks with mssparkutils.notebook.exit("done") is preferred.
- **Duplicated pipeline activities:** alerts_load and batch_job_2 existed twice, inside the True branch and outside it. Solution: the activities inside the True branch were deleted, and the branch was left empty.
- **Offline buffer overflow impossible with the guide windows:** with 3 to 60 minute windows and 30 second intervals, a buffer of 300 messages could never overflow. Solution: the overflow window was defined as the buffer capacity multiplied by the GPS interval plus 5 to 20 minutes (Section 6.8).
- **Late-arrival targets:** a fixed slowdown of 60 to 80 percent of speed cannot hit a late window of 10 to 60 minutes on routes of different length. Solution: the truck paces itself to a chosen arrival time (Section 6.7).

A troubleshooting reference for the pipeline is as follows.

- An activity fails after about 20 seconds and succeeds on retry: caused by Spark capacity. Increase the Wait activities (for example to 180 seconds), increase Retry, use an increasing delay, and close open sessions.
- batch_job_1 fails in the manifest cell: run_manifest.json or run_summary.json is missing. Check Files/evidence in lh_fleet.
- batch_job_2 fails with "table already exists": IF NOT EXISTS is missing in Cell 3. Add it.
- The If Condition fails or the Fail activity runs: Gate 1 returned FAIL. Open the batch_job_1 output and read the exitValue.
- The If Condition expression errors: the output.result.exitValue path or the activity name is wrong. Check the Output in Monitor and confirm that the name is batch_job_1.
- Bronze is slow or exceeds its timeout: each run covers 12 hours of data. Increase the timeout.

17. Project Limitations and Known Issues

- The Eventstreams must stay in the Running state at all times, because the pipeline reads only what has already been collected in KQL.
- The column vehicle_key in gold.fact_alerts is NULL on the first run, because alerts_load runs before batch_job_2. On later runs it is filled from the dim_vehicle of the previous run. An alternative is to move alerts_load after batch_job_2, as there is no reverse dependency.

- lateness_sec is tied to the run_manifest.json of a single run. If Bronze holds data from other runs, the values are inaccurate, and the notebook prints MISMATCH without stopping.
- Open interactive Spark sessions consume the same capacity as scheduled runs and should be closed before scheduled runs. The 120-second Wait activities are a workaround; if failures return, the Wait time or the Retry count should be increased.
- alerts_load reads the whole alert_events table every time (no watermark), which is acceptable at the current volume.
- When Gate 1 fails, Silver has already been written, and the pipeline stops before alerts_load and batch_job_2.
- verify_stream.py loads the whole message file into memory and should be used on the smoke and evidence profiles only, not on live_demo.
- The generator is a single process; if lag grows, A should be lowered.
- The evidence profile has 12 routes, whereas the project guide mentions about 40; additional city pairs can be added to ROUTE_DEFS in generate_reference_data.py.
- The generator writes the ground truth of late arrivals from the real arrival times; a truck that arrives late inside the grace period without being injected is not logged.
- With offline windows enabled, no data faults are applied to stored readings (Section 6.8).
- The evidence value of A is not final (Section 5.4), and the throughput test has to be repeated on the current setup.
- A mismatch between the mode in config.yaml and the reference data in Fabric produces wrong reconciliation and must be checked before every run.
- A connection string that has been pasted anywhere must be regenerated, and .env must never be committed.

18. Open Items and Future Work

The follow-up checklist from the orchestration documentation contains the following open items.

- Confirm that the first scheduled run started on time.
- Review the duration of Bronze and batch_job_1 and adjust the timeouts if needed.
- Add the Failure notifications email address.
- Replace session.stop() with notebook.exit in the remaining notebooks.
- Check the Spark capacity message in the Output of the failed attempts from the first run, before the Wait activities were added.
- Watch the first few scheduled runs to confirm that the 120-second Waits are sufficient, since one clean run is not proof.
- Make a final decision on the order of alerts_load and batch_job_2 (the vehicle_key issue).

The Phase 1 documentation additionally lists the following open items.

- Complete the local verification of the offline windows (P1-12).
- Record the results of the Fabric checks of Section 14.4.
- Decide the final value of the acceleration factor A and repeat the throughput test on the current setup.
- Test the Power BI refresh and Direct Lake lag.
- Generate and run the full 5,000-vehicle reference data as the final step (Section 15.3).

The real-time documentation additionally leaves the following items to be recorded.

- The refresh settings of the dashboard rtd_fleet_live.
- The selected schedule frequency of the pipeline pl_rt_alert_writer.
- The results of the parity test, precision and recall (Section 10.6).

19. Conclusion

The project has established a reproducible fleet telemetry simulation with a documented ground truth, a Bronze layer of raw text tables in Eventhouse, a validated Silver layer with a dead-letter queue and a quality gate, a real-time KQL layer that validates telemetry, detects cold-chain, idling and route-deviation alerts and feeds a two-page live dashboard, a Gold layer organised as a galaxy schema, and a scheduled Data Factory pipeline that runs the batch steps in order. On the smoke dataset, every Silver table balances against Bronze, and the pipeline has completed successfully in repeated runs. The remaining work consists of the open items listed in Section 18, in particular the Fabric-side verification checks, the final choice of the acceleration factor, and the final evidence run on the full reference data before the trial capacity ends.